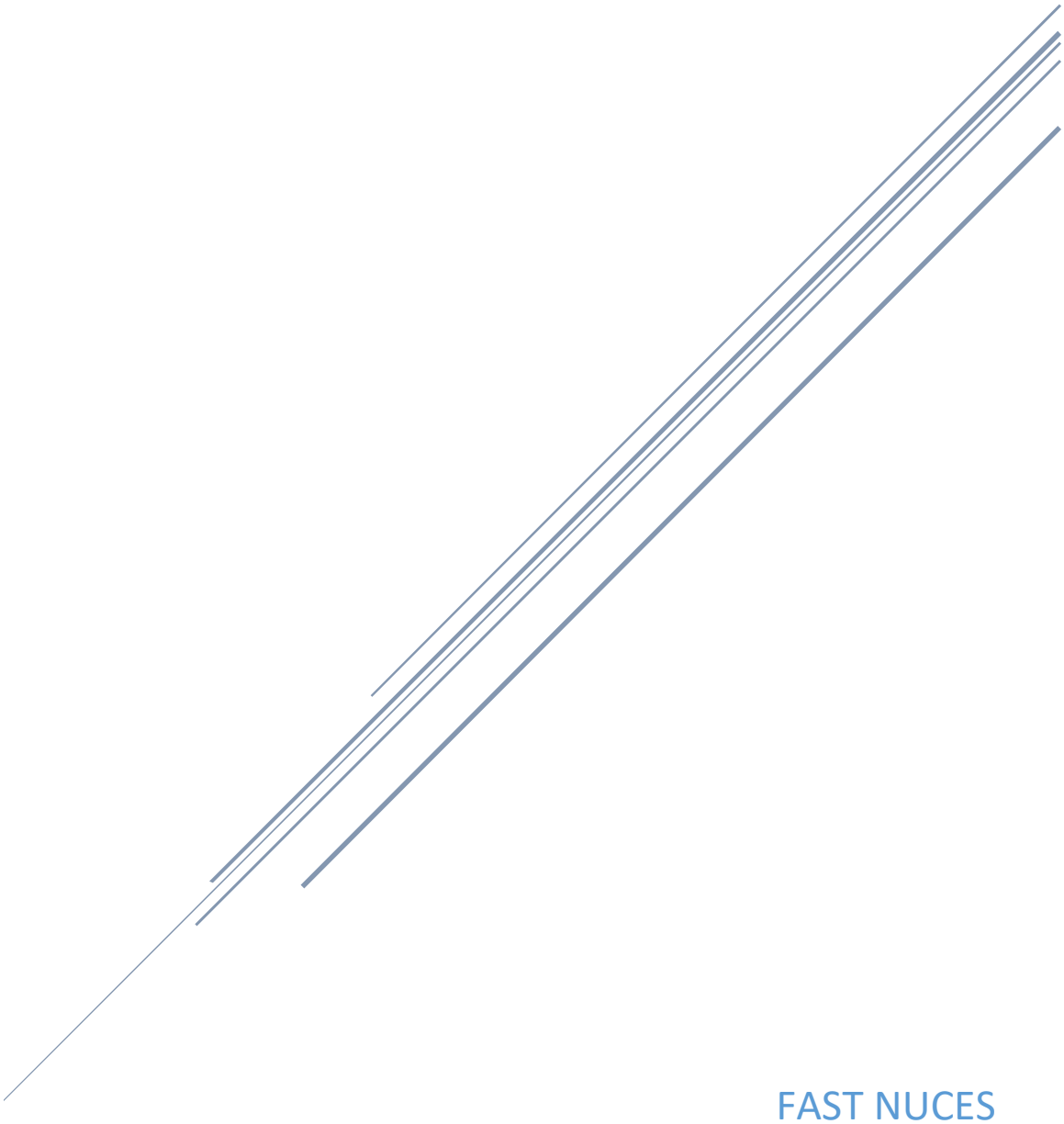
MUHAMMAD HUSSAIN
26K-0615

PF ASSIGNMENT 1

SUBMISSION DATE : 17-SEP-26



FAST NUCES
PF

NAME: MUHAMMAD HUSSAIN (A) Date: 11-Sept-26

ROLL NO: 26K-0615

PART A:

Question 1:

Solution:

Algorithm:

1. Start
2. Read a number N from the user.
3. While N is greater than or equal to 1:
   - Display N.
   - Decreased N by 1.
4. Stop.

Pseudocode:

```
START
INPUT N

WHILE N>=1
        OUTPUT N
        N=N-1
END WHILE

END
```

Q2: Solution:

Algorithm:

1. Start.
2. Read a years from the users.
3. Check if the year is divisible by 4 and not divisible by 100, or if it is divisible by 400.
4. If the condition is true, display "LEAP YEAR"; otherwise, display "NOT A LEAP YEAR"
5. STOP.

Page No.

Date: _____

Pseudocode:

Start

START
INPUT YEAR

IF (YEAR % 4 == 0 AND YEAR % 100 != 0) OR YEAR % 400 == 0 THEN
    OUTPUT "LEAP YEAR"
ELSE
    OUTPUT "NOT A LEAP YEAR"
END IF

END.

Question 3:

Solution:

Algorithm:

1. Start
2. Read a number from the user.
3. Initialize Count to 0.
4. Divide the number by 10 until the number becomes zero, and increment COUNT by 1 after each division.
5. Display COUNT.
6. If COUNT is 1, display "SINGLE-DIGIT".
7. Else if COUNT is 2, display "DOUBLE-DIGIT".
8. Otherwise, display "TRIPLE-DIGIT OR MORE".
9. Stop.

Pseudocode:

```
START
INPUT NUMBER

COUNT = 0

WHILE NUMBER > 0
        NUMBER = NUMBER / 10
        COUNT = COUNT + 1
END WHILE

OUTPUT COUNT

IF COUNT == 1 THEN
    OUTPUT "SINGLE-DIGIT"
ELSE IF COUNT == 2 THEN
    OUTPUT "DOUBLE-DIGIT"
ELSE
    OUTPUT "TRIPLE-DIGIT" OR MORE"
END IF

END
```

(Q4) Solution:

Algorithm:

1. Start
2. Initialize I to 1.
3. WHILE I is less than or equal to 10:
   - Check if I is not a multiple of 3.
   - If it is not a multiple of 3:
     - Initialize J to 1.
     - WHILE J is less than or equal

to 10.
- Display I x J
- Increase J by 1.
- Increase I by 1.

4. Stop.

Pseudocode:

```
START

I = 1

WHILE I <= 10
    IF I%3 != 0 THEN
        J = 1
        WHILE J <= 10
            OUTPUT I*J
            J = J + 1
        END WHILE
    END IF
    I = I + 1
END WHILE
END
```

(Q5) Solution:

Algorithm:

1. Start.
2. Read N from the use
3. While N has two or more digits:
   - Initialize SUM to 0.
   - While N is greater than 0:
     - Find the last digit of N.
     - Add the digit to SUM.

Date: _____

- Remove the last digit from N.
- Display SUM.
- Set N equal to SUM.

4. Stop.

Pseudocode:

```
START
INPUT N

WHILE N>=10
    SUM=0

    WHILE N >0
        DIGIT = N % 10
        N=N/10
        SUM = SUM + DIGIT
    END WHILE

    OUTPUT SUM
    N = SUM
END WHILE
END.
```

Source Code Question 1 Hotel Booking System

```
#include<stdio.h>


#define PEAK_STANDARD_RATE 5000

#define PEAK_DELUXE_RATE 8000

#define PEAK_SUITE_RATE 12000

#define OFF_STANDARD_RATE 3000

#define OFF_DELUXE_RATE 5000

#define OFF_SUITE_RATE 8000


int main(){

printf("\n=================================================================
==\n");

    printf("\t\tWELCOME TO HOTEL 5-STAR");

printf("\n=================================================================
==\n");




    int season,room_type,no_night;

    double discount, customer_bill,hotel_revenue = 0;

    // double rate;

        printf("\nWelcome to Our Hotel.\nIn order to accommodate you please
answer the following question:\n\n");


        label2:

        printf("Select Season: (1). PEAK SEASON\t(2). OFF SEASON\t : ");

        scanf("%d", &season);


        if(season <=0 || season > 2){
```

```
            printf("\nInvalid number entered[!].Try Again\n");
            goto label2;
        }

        label3:
        printf("\nSelect Room type : (1). STANDARD\t(2). DELUXE\t(3). SUITE :
");
        scanf("%d",&room_type);

        if(room_type<=0 || room_type > 3){
            printf("\nInvalid number entered[!].Try Again");
            goto label3;
        }

        label4:
        printf("\nEnter NO OF NIGHTS you will stay : ");
        scanf("%d", &no_night);

        if(no_night<=0){
            printf("\nInvalid number entered[!].Try Again\n");
            goto label4;
        }

        if(season == 1){
            if(room_type == 1) {
                customer_bill = PEAK_STANDARD_RATE * no_night;
            } else if (room_type == 2) {
                customer_bill = PEAK_DELUXE_RATE * no_night;
            } else {
                customer_bill = PEAK_SUITE_RATE * no_night;
            }
        } else {
```

```
            if(room_type == 1) {
                customer_bill = OFF_STANDARD_RATE * no_night;
            } else if (room_type == 2) {
                customer_bill = OFF_DELUXE_RATE * no_night;
            } else {
                customer_bill = OFF_SUITE_RATE * no_night;
            }
        }

        if(no_night>7){
            discount = customer_bill  * 0.15;
            customer_bill = customer_bill - discount;
            printf("\n[+] Long-stay discount applied (15%% off: RS.%.2lf)\n",
discount);
            printf("\nYour Bill for %d NO OF NIGHTS in Room type %d  is
RS.%.2lf\n", no_night, room_type, customer_bill);
        } else {
            printf("\nYour Bill for %d NO OF NIGHTS in Room type %d is
RS.%.2lf\n", no_night, room_type, customer_bill);
        }
        hotel_revenue = hotel_revenue + customer_bill;




printf("\n=================================================================
==\n");
    printf("\tHOTEL TOTAL REVENUE: RS.%.2lf\n", hotel_revenue);

printf("=================================================================
\n");
    return 0 ;
}
```

OUTPUT SCREENSHOT:

```
PS G:\Programming\Pf_a1> gcc HotelBookingSystem.c -o a
PS G:\Programming\Pf_a1> ./a

======================================================================
                WELCOME TO HOTEL 5-STAR
======================================================================

Welcome to Our Hotel.
In order to accommodate you please answer the following question:

Select Season: (1). PEAK SEASON (2). OFF SEASON  : 2

Select Room type : (1). STANDARD        (2). DELUXE     (3). SUITE : 3

Enter NO OF NIGHTS you will stay : 9

[+] Long-stay discount applied (15% off: RS.10800.00)

Your Bill for 9 NO OF NIGHTS in Room type 3  is RS.61200.00

======================================================================
        HOTEL TOTAL REVENUE: RS.61200.00
======================================================================
PS G:\Programming\Pf_a1>
```

no guests)

PART B

Question 1: Hotel Booking System

Problem Analysis Chart (PAC):

Data	Processing	Output
N-number of guests.	bill = rate x no.night	Bill for each guest.
season (1=Peak, 2=Off).	if no_night > 7 : discount = bill x 0.15; bill = bill - discount	Total revenue for the hotel.
room type (1=Standard, 2=Deluxe, 3=Suite)	hotel_revenue = hotel_revenue + bill (accumulated over N guests)	
no_night - nights stayed.		
Rate table : Peak (5000/8000/12000) Off (3000/5000/8000)		

Algorithm

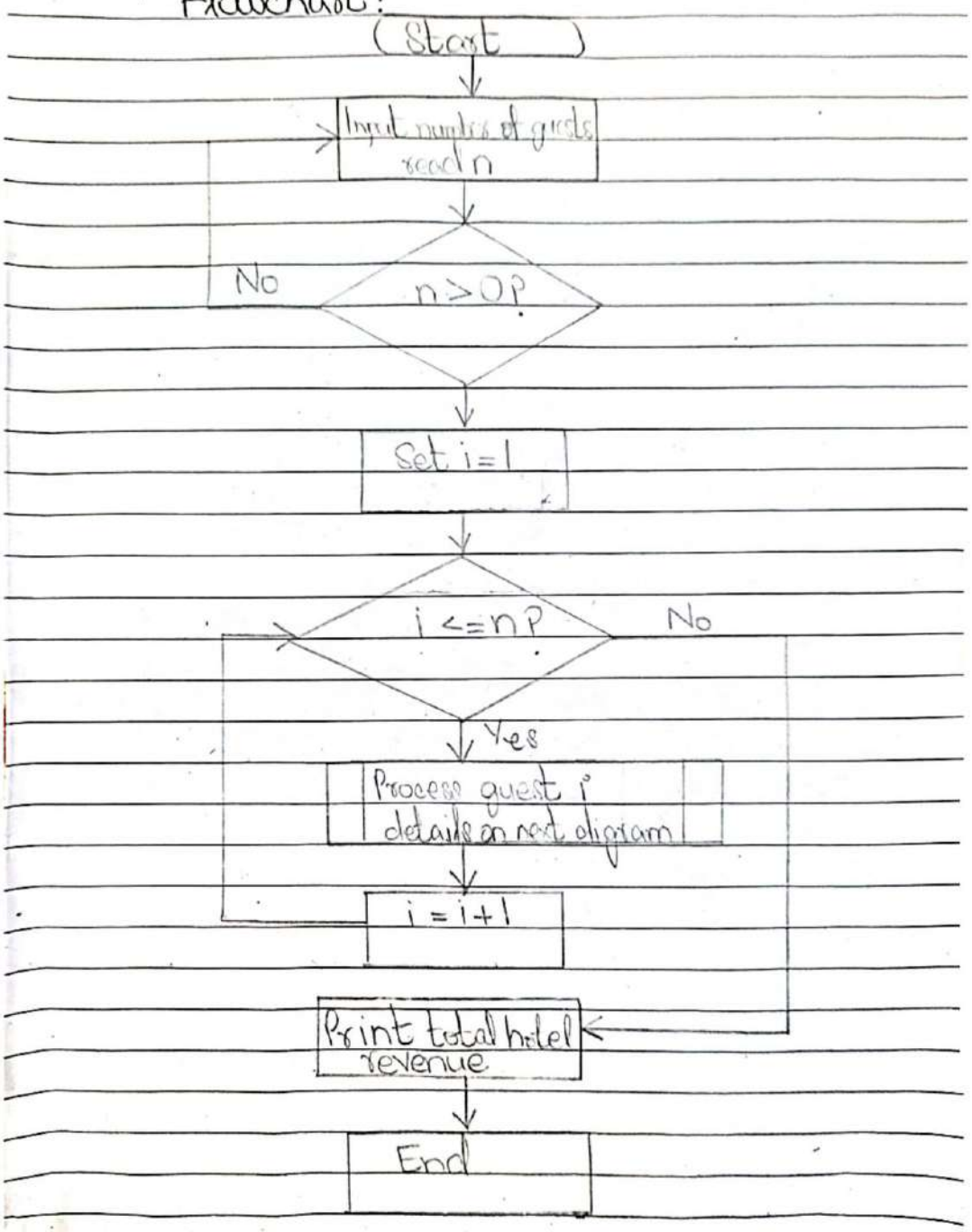
1. Start
2. Display the welcome banner
3. Input N (total number of guests)
4. If N <= 0, show an error message and repeat Step 3.
5. Initialize i = 1 and hotel_revenue = 0.
6. While i <= N, repeat Step 7-14 for each guest.
7. Input, season (1 = Peak, 2 = Off); repeat until it's 1 or 2.
8. Input room_type (1 = Standard, 2 = Deluxe, 3 = Suite); repeat until its 1-3.
9. Input no_night; repeat until it's greater than 0.
10. Look up the per night rate for the chosen season + room type combination and compute customer_bill = rate x no_night
11. If no_night > 7 calculate discount = customer_bill x 0.15 and subtract it from customer_bill.
12. Display the bill (and discount, if applied)
13. Add customer_bill to hotel_revenue.
14. Increment i by 1.
15. After the loop finished, display hotel_revenue.

Page No.

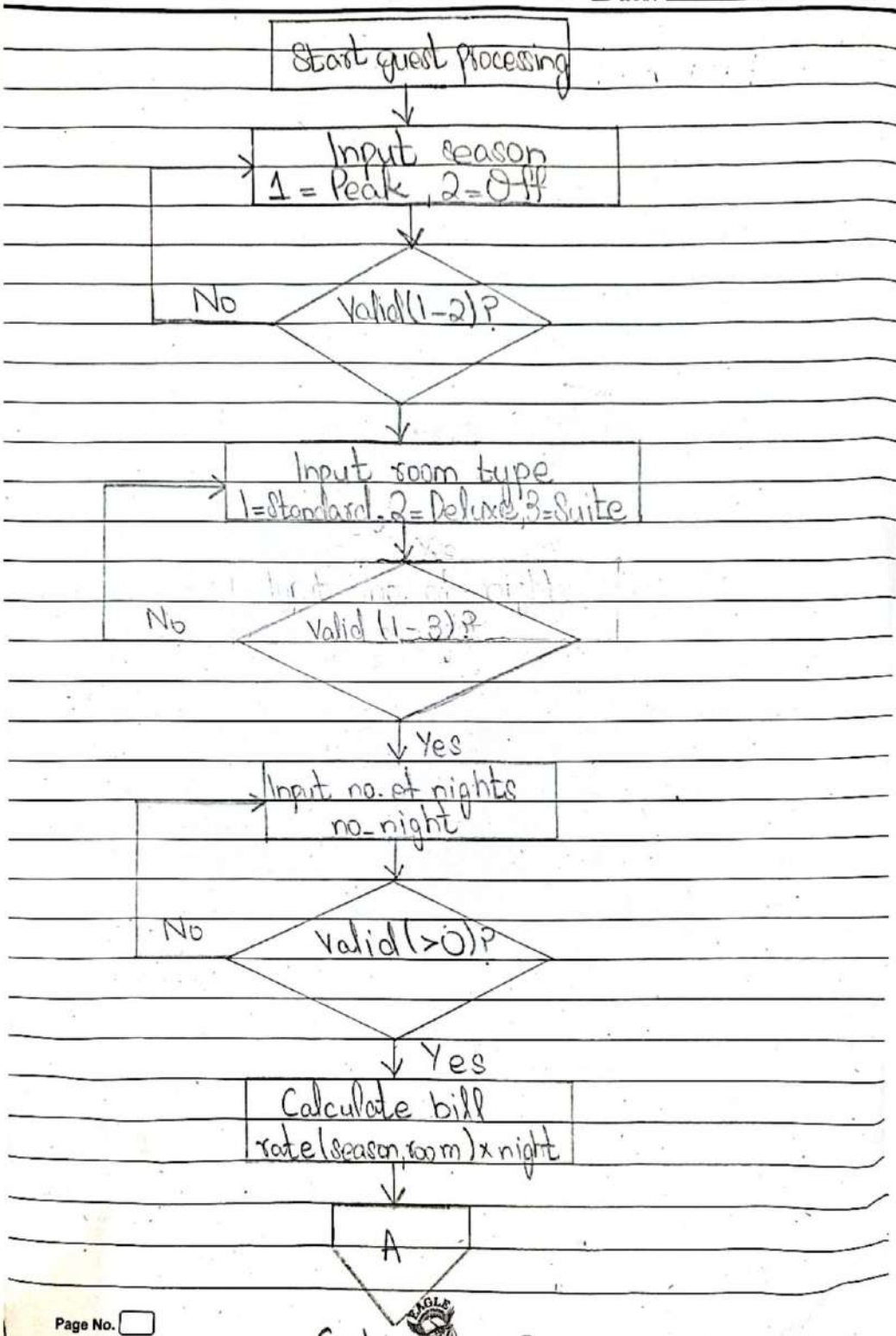
Date: _____

16. Stop.

Flowchart:

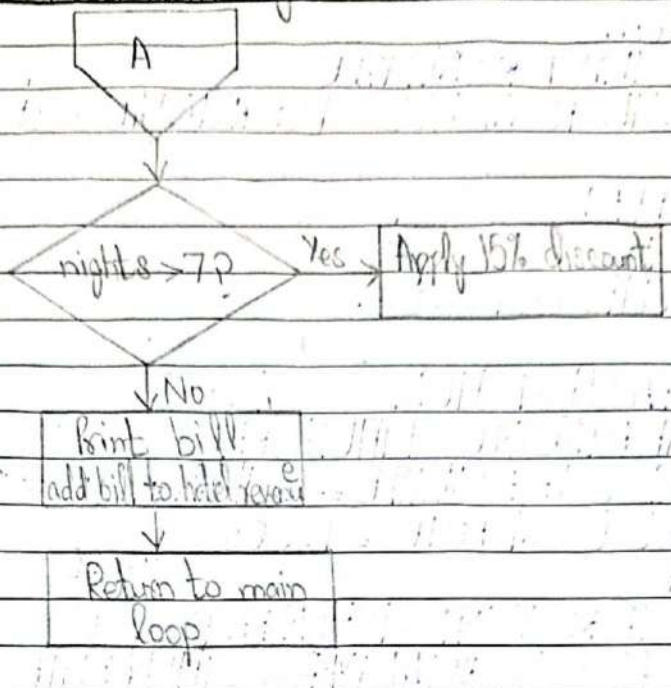


Date: _____



Continued on Page 2.

Continued from Page 1 Date: _____



Pseudocode:

```
START
    DISPLAY WELCOME MESSAGE
    REPEAT
        INPUT N
        IF N<=0 THEN DISPLAY "INVALID UNTIL
N>0

    SET hotel_revenue = 0

FOR i = 1 TO N DO
        REPEAT
            INPUT SEASON
        UNTIL SEASON=1 OR SEASON=2
```

Date: _____

```
REPEAT
    INPUT ROOM_TYPE
UNTIL ROOM_TYPE = 1 OR ROOM_TYPE = 2 OR ROOM_TYPE = 3

REPEAT
    INPUT no_night
UNTIL no_night > 0

IF SEASON = 1 THEN          // PEAK
    IF ROOM_TYPE = 1 THEN RATE = 5000
    ELSE IF ROOM_TYPE = 2 THEN RATE = 8000
    ELSE RATE = 12000
ELSE                        // Off
    IF ROOM_TYPE = 1 THEN RATE = 3000
    ELSE IF ROOM_TYPE = 2 THEN RATE = 5000
    ELSE rate = 8000
ENDIF
    bill = rate * no_night

IF no_night > 7 THEN
        discount = bill * 0.15
        bill = bill - discount
        DISPLAY "DISCOUNT APPLIED"
ENDIF
    DISPLAY "BILL FOR GUEST", i, " =", bill
    hotel_revenue = hotel_revenue + bill

ENDFOR

DISPLAY "TOTAL HOTEL REVENUE = ", hotel_revenue

END.
```

IPO

Date:

Input	Processing	Module	Output
N(total_guests)	Enter N; validate N > 0	1000	Bill per guest
season	For each guest (i = 1 to N): enter season; validate (1-2)	2000	Hotel total revenue
room_type	Enter room type; validate (1-3)	3000	
no_night	Enter no night; validate > 0	4000	
	Select rate by season and room_type; bill = rate × no night	5000	
	if no_night ≥ 7; discount = 15% of bill; bill = bill - discount	6000	
	Display guest's bill; add bill to hotel_revenue	7000	
	Display hotel_revenue	8000	
	end	9000	

Source Code Question 02 Elevator Simulation

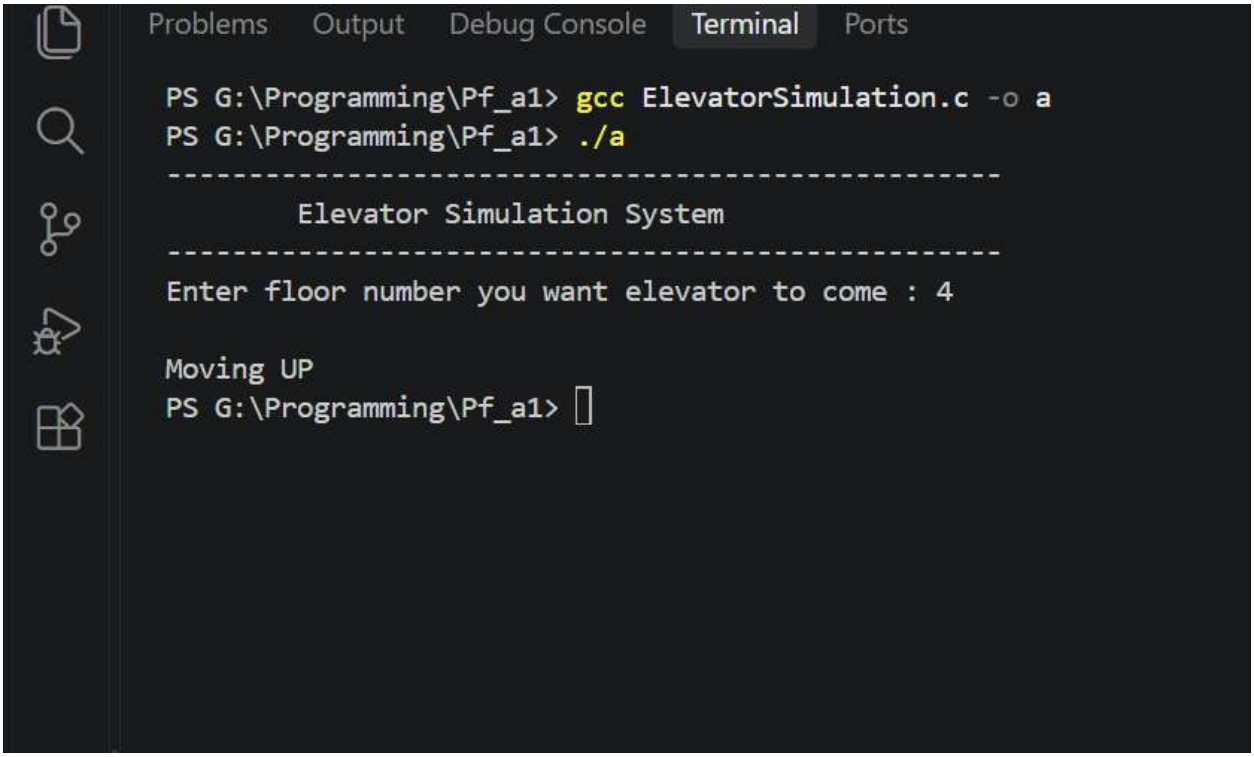
```
#include<stdio.h>
int main(){
printf("-----------------------------------------------------");
printf("\n\tElevator Simulation System\n");
printf("-----------------------------------------------------");

int current_floor = 0,total_requests;
int floor_request;

label1:
printf("\nEnter floor number you want elevator to come : ");
scanf("%d", &floor_request);
if(floor_request<0) {
    printf("\nInvalid Input[!!!].Enter again");
    goto label1;
}

    if(current_floor < floor_request) {
        printf("\nMoving UP");
    } else if (current_floor > floor_request) {
        printf("\nMoving Down");
    } else {
        printf("\nDoors Opening");
    }
    current_floor = floor_request;
    return 0;
}
```

OUTPUT SCREENSHOT:



Question 2: Elevator Simulation System

Algorithm:

1. Start
2. Display welcome banner
3. Set current_floor = 0, i = 0
4. Input total_requests; if ≤ 0, show error and repeat.
5. While i < total_requests, repeat step 6-9.
6. Input floor_request (the next requested floor).
7. If current_floor < floor_request → print "Moving Up"
   Else if current_floor > floor_request → print "Moving DOWN".
   Else → print "Doors Opening".
8. Set current_floor = floor_request
9. Increment i by 1
10. Print "All requests are successfully done".
11. Stop.

Pseudocode

```
START
  DISPLAY WELCOME BANNER
  current_floor = 0
  i = 0

  REPEAT
      INPUT total_requests
      IF total_requests <= 0 THEN DISPLAY
"Invalid number of requests".
  UNTIL total_requests > 0

  WHILE i < total_requests DO
        INPUT floor_request
```

Date:

```
IF current_floor < floor_request THEN
        DISPLAY "MOVING UP"
ELSE IF current_floor > floor_request THEN
        DISPLAY "MOVING DOWN"
ELSE
        DISPLAY "DOORS OPENING"
ENDIF
    current_floor = floor_request
    i = i + 1
ENDWHILE

DISPLAY "ALL REQUESTS ARE SUCCESSFULLY
        DONE"
END.
```

Input Process Output Chart (IPO CHART).

Input	Processing	Module	Output
total_requestes	Enter total requestes.	1000	
	If total_requestes <= 0, go to 1000	1100	
	Set current_floor=0, i=0	2000	
	If i<total_requestes, go to 4000, else go to 8000	3000	
floor_request	Enter floor request	4000	

Page No.

Date:

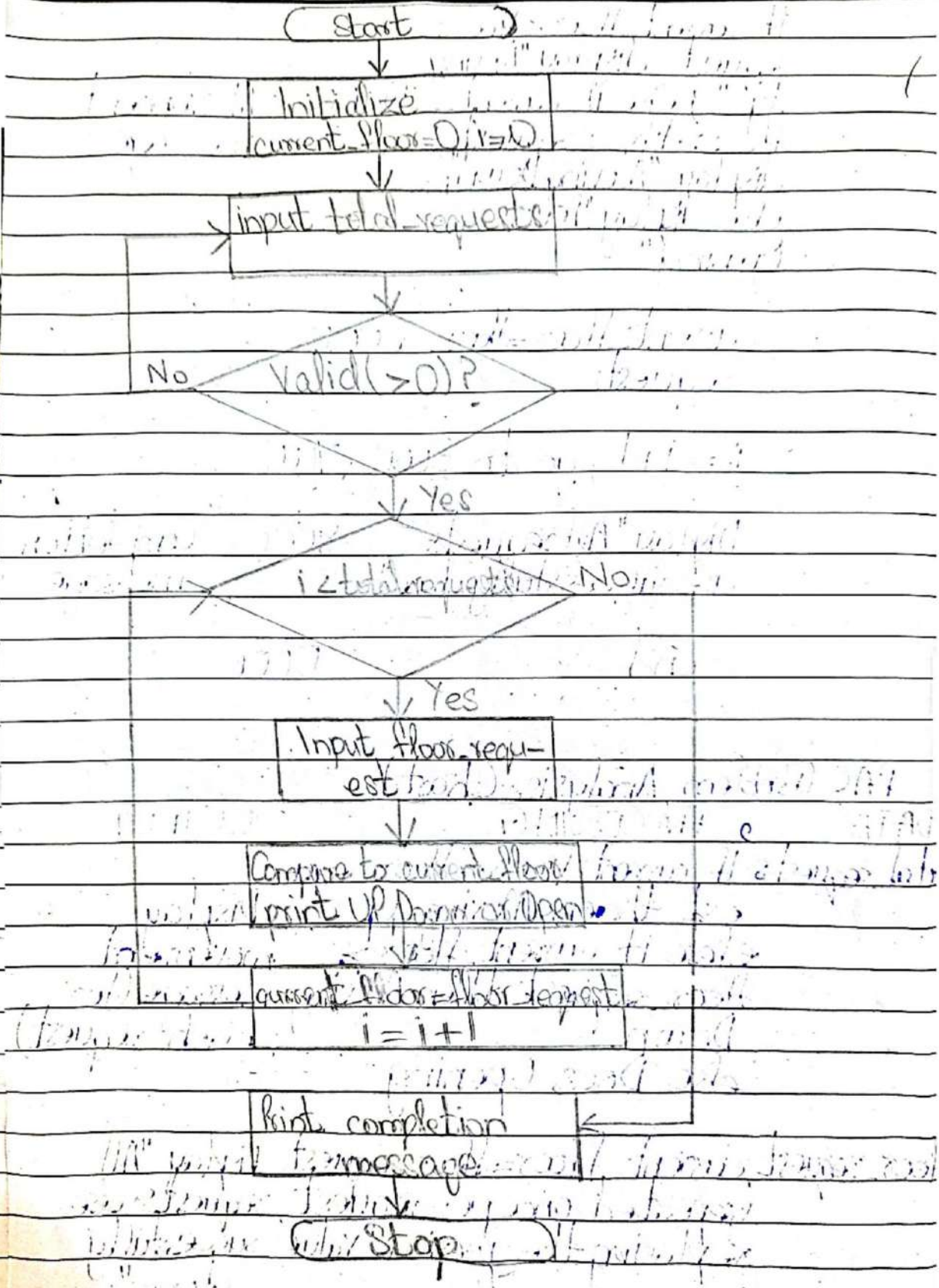
If current_floor < floor_request, display "Moving Up"; else if current_floor > floor_request, display "Moving Down"; else display "Doors Opening"	5000	Movement message
current_floor = floor_request	6000	
i = i+1, go to 3000	7000	
Display "All requests are successfully done"	8000	Completion message
end	0000	

PAC (Problem Analysis Chart)

DATA	PROCESSING	OUTPUT
total_requests	If current_floor < floor_request then Moving Up. else if current_floor > floor_request then Moving Down. else Doors Opening	Display movement message (for each request)
floor_request	current_floor = floor_request (repeated once per request, replacing the previous value	Display "All requests are successfully done"

FLOWCHART

Date: _____



Source Code Question Question 03 Class Result Processing

```
#include<stdio.h>

int main(){

int total_students, i = 0;

float maths,phy,eng,pf,icp,avg,sum;


printf("+=+=+=
```

```
    printf("\nInvalid Marks Entered[!].Try Again Harder[!]\n");
    goto label2;
}

label3:
    printf("\nEnter the marks of Functional English  : ");
    scanf("%f", &eng);

if(eng<0 || eng>100) {
    printf("\nInvalid Marks Entered[!].Try Again Harder[!]\n");
    goto label3;
}

label4:
    printf("\nEnter the marks of Programming Fundamentals  : ");
    scanf("%f", &pf);

if(pf<0 || pf>100) {
    printf("\nInvalid Marks Entered[!].Try Again Harder[!]\n");
    goto label4;
}

label5:
    printf("\nEnter the marks of Ideology And Constitution Of Pakitan  : ");
    scanf("%f", &icp);

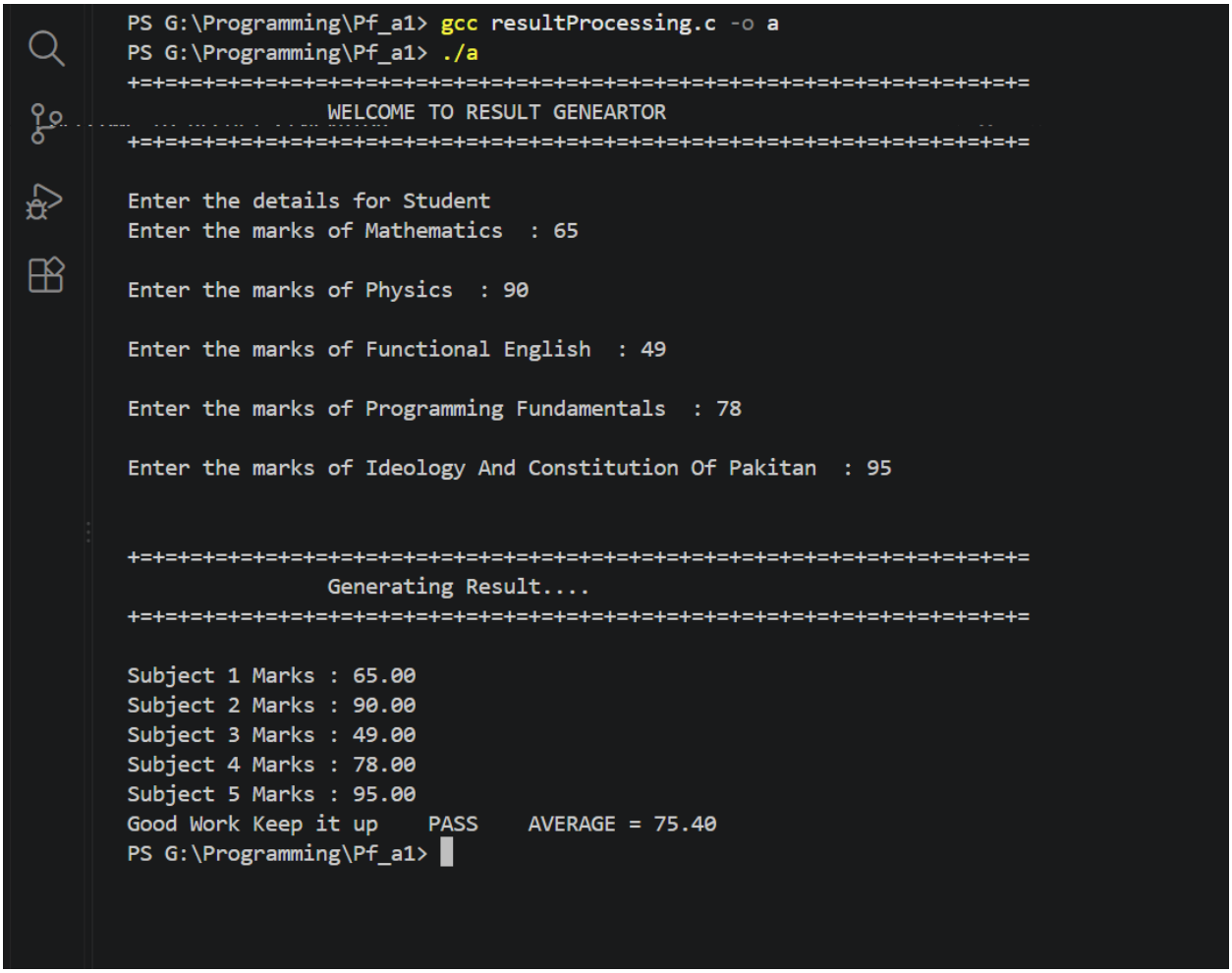
if(icp<0 || icp>100) {
    printf("\nInvalid Marks Entered[!].Try Again Harder[!]\n");
    goto label5;
}

sum = maths + phy + eng + pf + icp;
```

```
avg = sum / 5;

printf("\n\n+=+=+=
```

OUTPUT SCREENSHOT:


PS G:\Programming\Pf_a1> gcc resultProcessing.c -o a
PS G:\Programming\Pf_a1> ./a
+=+=+=

Date:

Q3: Class Result Processing

Algorithm:

1. Start.
2. Display welcome message.
3. Input total students.
4. Set i = 0.
5. While i < total_students, repeat steps 6-14.
6. Input maths; if not b/w 0-100, show error and repeat.
7. Input phy; if not b/w 0-100 show error and repeat.
8. Input eng; if not b/w 0-100, show error and repeat.
9. Input pf; if not b/w 0-100, show error and repeat.
10. Input icp; if not b/w 0-100 show error and repeat.
11. Compute sum = maths + phy + eng + pf + icp and avg = sum/5.
12. Print all 5 subject marks.
13. Check if any subject is below 33, display Subject Defiency.
14. Decide and print the result:
    - if any subject < 33 → "Fail due to subject defiency"
    - Else if avg >= 80 → "Distinction"
    - Else if avg >= 60 → "Pass"
    - Else → "Fail due to average less than 60"
15. Increment i.
16. Stop.

IPO Chart. Date:

Input	Processing	Module	Outputs
total students	Enter total students	1000	
	set i = 0	2000	
	if i<total students, goto 4000, else go to 9500		
maths	Enter maths	4000	
	if not in [0,100], goto 4000	4100	
phy	Enter phy	5000	
	if not in [0,100], goto 5000	5100	
eng	Enter eng	6000	
	if not in [0-100], goto 6000	6100	
pf	Enter pf	7000	
	if not in [0,100], goto 7000	7100	
icp	Enter icp	8000	
	if not in [0,100] goto 8000	8100	
	sum=maths+phy+eng+pf+icp;	8200	
	avg=sum/5		
	Display all 5 marks	8300	Marksheet
	Determine all sub pass(any mark<0<33?)	8400	
	Display result: fail(deficiency)/ Distinction/Pass/fail(avg<60)	8500	Result label
	i=i+1 goto 3000	8600	
	end	0000	

Page No.

PAC (Problem Analysis Chart) Date: _____

Data	Processing	Output
Total Students ma maths, phy, eng, pf, icp	sum = maths + phy + eng + pf + icp. avg = sum / 5. deficiency = TRUE if any mark < 33 result = If deficiency Then Fail Else if avg >= 80 Then "Distinction" Else if avg >= 60 Then "Pass" Else "Fail"	Display all 5 subject mark Display result label

PSEUDOCODE

```
START
  DISPLAY WELCOME MESSAGE
  INPUT TOTAL_STUDENTS
  Set i = 0

WHILE i < TOTAL_STUDENTS DO
  REPEAT INPUT maths UNTIL 0 <= maths <= 100
  REPEAT INPUT phy UNTIL 0 <= phy <= 100
  REPEAT INPUT eng UNTIL 0 <= eng <= 100
  REPEAT INPUT pf UNTIL 0 <= pf <= 100
  REPEAT INPUT icp UNTIL 0 <= icp <= 100

  sum = maths + phy + eng + icp + pf
  avg = sum / 5.

  DISPLAY maths, phy, eng, pf, icp.
```

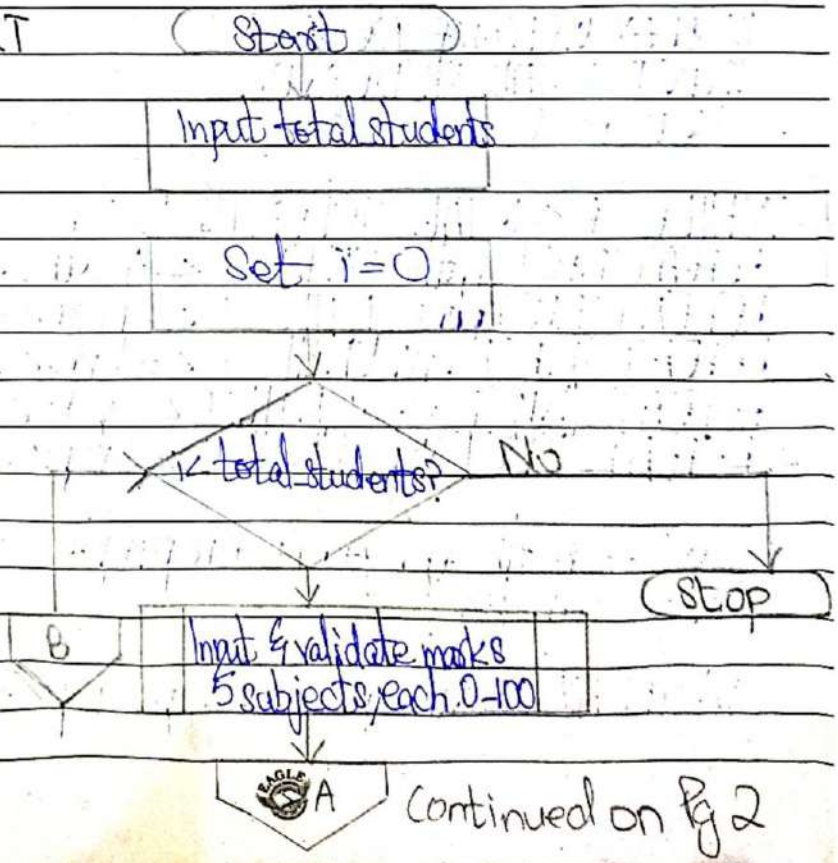
Date:

```
IF maths < 33 OR phy < 33 OR eng < 33 OR
   pf < 33 OR icp < 33 THEN
    DISPLAY "FAIL DUE TO SUBJECT DEFICIENCY"
ELSE IF avg >= 80 THEN
    DISPLAY "DISTINCTION"
ELSE IF avg >= 60 THEN
    DISPLAY "PASS"
ELSE
    DISPLAY "Fail DUE TO AVG < 60"

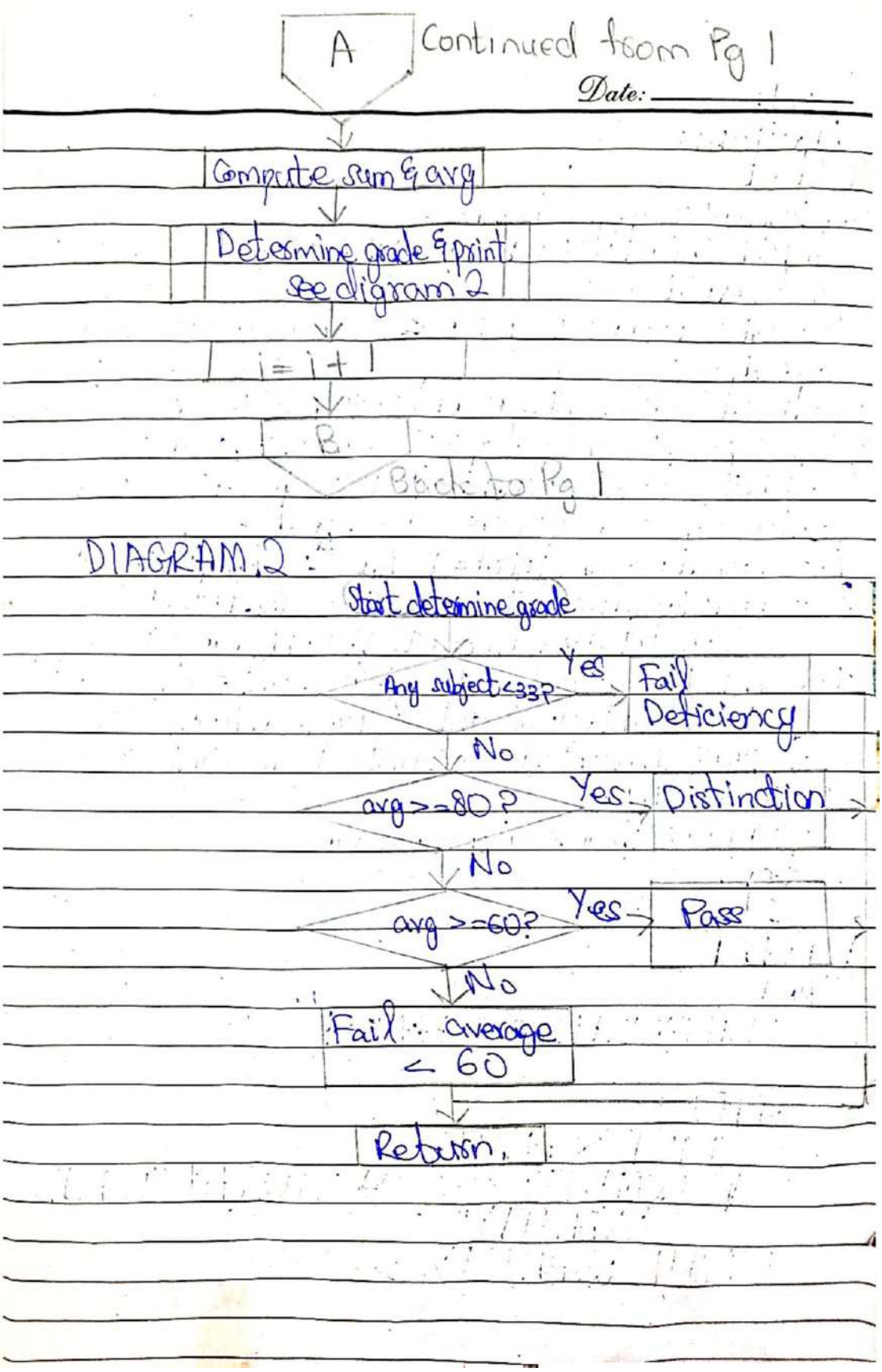
ENDIF

i = i + 1
ENDWHILE
END.
```

FLOWCHART



Continued on Pg 2


A
Continued from Pg 1
Date:
Compute sum & avg
Determine grade & print
see digram 2
i = i + 1
B
Back to Pg 1
DIAGRAM 2 :
Start determine grade
Any subject < 33?
Yes
Fail
Deficiency
No
avg >= 80?
Yes
Distinction
No
avg >= 60?
Yes
Pass
No
Fail : average
< 60
Return.

Source Code Question Question 04 Online Shopping Bill Calculator

```
#include<stdio.h>
int main(){
    printf("================================================\n");
    printf("           E-SHOP SMART BILLING SYSTEM         \n");
    printf("================================================\n");
    printf("  Welcome! We are thrilled to have you shop with us.\n");
    printf("  Let's calculate your order total in a few easy steps.\n");
    printf("------------------------------------------------\n\n");

    int quantity;
    double price;
    float discount_percentage;
    float tax_percentage;

    label1:
    printf("\nEnter the quantity of products purchased : ");
    scanf("%d", &quantity);
    if(quantity<=0) {
        printf("\nInvalid Number Of Products Entered[!]\n");
        goto label1;
    }

    label2:
    printf("\nEnter the price per item : ");
    scanf("%lf", &price);
    if(price<=0) {
```

```
        printf("\nInvalid Price Entered[!]\n");
        goto label2;
    }

    label3:
    printf("\nEnter the discount percentage (%%) : ");
    scanf("%f", &discount_percentage);
    if(discount_percentage<=0 || discount_percentage>=20) {
        printf("\nInvalid Discount Percentage Entered[!]\n");
        goto label3;
    }

    label4:
    printf("\nEnter the Tax percentage (%%) : ");
    scanf("%f", &tax_percentage);
    if(tax_percentage<=0 || tax_percentage>=15) {
        printf("\nInvalid Tax Percentage Entered[!]\n");
        goto label4;
    }

    double sub_total = quantity * price;
    double discounted_amount = sub_total - (sub_total * discount_percentage)
/ 100.0;
    double final_bill = discounted_amount + (discounted_amount *
tax_percentage)/100;

    printf("-----------------------------------------\n");
    printf("\n\tGENERATING BILL....");
    printf("\n-----------------------------------------\n");

    printf("\nYour subtotal is:\t\tRs.%.2lf", sub_total);
    printf("\nYour Discounted Amount is:\tRs.%.2lf", discounted_amount);
    printf("\n-----------------------------------------");
```

```
    printf("\nYour Final Bill is:\tRs.%.2lf", final_bill);

    printf("\n-------------------------------------------------\n");
    printf("         Thank you for shopping with us!      \n");
    printf("=================================================\n");
    printf("  Your transaction has been successfully processed.\n");
    printf("  Have a wonderful day! Goodbye!                  \n");
    printf("=================================================\n");

    return 0;
}
```

OUTPUT ON NEXT PAGE

OUTPUT SCREENSHOT:

```
PS G:\Programming\Pf_a1> gcc shoppingBill.c -o a
PS G:\Programming\Pf_a1> ./a
================================================
           E-SHOP SMART BILLING SYSTEM
================================================
  Welcome! We are thrilled to have you shop with us.
  Let's calculate your order total in a few easy steps.
------------------------------------------------


Enter the quantity of products purchased : 3

Enter the price per item : 400

Enter the discount percentage (%) : 10

Enter the Tax percentage (%) : 5
------------------------------------------

        GENERATING BILL....
------------------------------------------

Your subtotal is:               Rs.1200.00
Your Discounted Amount is:      Rs.1080.00
------------------------------------------------
Your Final Bill is:     Rs.1134.00
------------------------------------------------
         Thank you for shopping with us!
================================================
  Your transaction has been successfully processed.
  Have a wonderful day! Goodbye!
================================================
PS G:\Programming\Pf_a1>
```

Q4: Online Shopping bill Calculator Date: _____

Algorithm:

1. Start
2. Display welcome banner
3. Input quantity; if quantity ≤ 0, show error and repeat.
4. Input price; if price ≤ 0, show error and repeat.
5. Input discount_percentage; if its ≤ 0 or > 20 show error and repeat.
6. Input tax percentage; if its ≤ 0 or ≥ 15, show error and repeat.
7. Compute sub_total = price * quantity.
8. Compute discount_amount = sub_total - (sub_total x discount_percentage / 100)
9. Compute final_bill = discounted_amount + (discounted_amount x tax_percentage / 100).
10. Display sub_total, discounted_amount and final_bill.
11. Display thank you message
12. Stop.

PSEUDOE

START

DISPLAY WELCOME BANNER

REPEAT

INPUT QUANTITY

IF QUANTITY <=0 THEN DISPLAY "INVALID QUANTITY"

UNTIL QUANTITY > 0

Date:

```
REPEAT
    INPUT PRICE
    IF PRICE <=0 THEN DISPLAY "INVALID
    PRICE".
UNTIL PRICE >0

    REPEAT
        INPUT DISCOUNT_PERCENTAGE
        IF DISCOUNT_PERCENTAGE <= 0 OR
        DISCOUNT_PERCENTAGE >=20
        THEN DISPLAY "INVALID DISCOUNT".

    UNTIL DISCOUNT_PERCENTAGE >0 AND
          DISCOUNT_PERCENTAGE <20

    REPEAT
        INPUT tax_Percentage
        IF tax_Percentage <=0 OR tax_Percentag
        e >=15 THEN DISPLAY "INVALID TAX".

    UNTIL tax_Percentage >0 AND tax_Percentage
          <15.

    sub_total = quantity * Price
    discanted_amount = subtotal - (sub_total *
    discount_percentage/100)

    final_bill = discounted_amount + (discounted
        _amount * tax_percentage /100)

    DISPLAY sub_total, discounted_amount,
            final_bill.

END
```

Page No.

IPO CHART

Date:

Input	Processing	Module	Output
quantity	Enter quantity	1000	
	If quantity <=0, goto 1000	1100	
	Enter price	2000	
	If price <=0, goto 2000	2100	
discount percentage	Enter discount_percentage	3000	
	If discount_percentage <=0 or >=100 goto 3000	3100	
tax_percentage	Enter tax_percentage	4000	
	If tax_percentage <=0 or >=15 goto 4000	4100	
	sub_total = quantity * price	5000	
	discounted_amount = sub_total - (sub_total * discount_percentage / 100)	6000	
	final bill = discounted_amount + (discounted_amount * tax_percentage / 100)	7000	
	Display sub_total, discounted_amount, final bill	8000	final bill
	Display thank you message end	9000 9000	Thank you msg

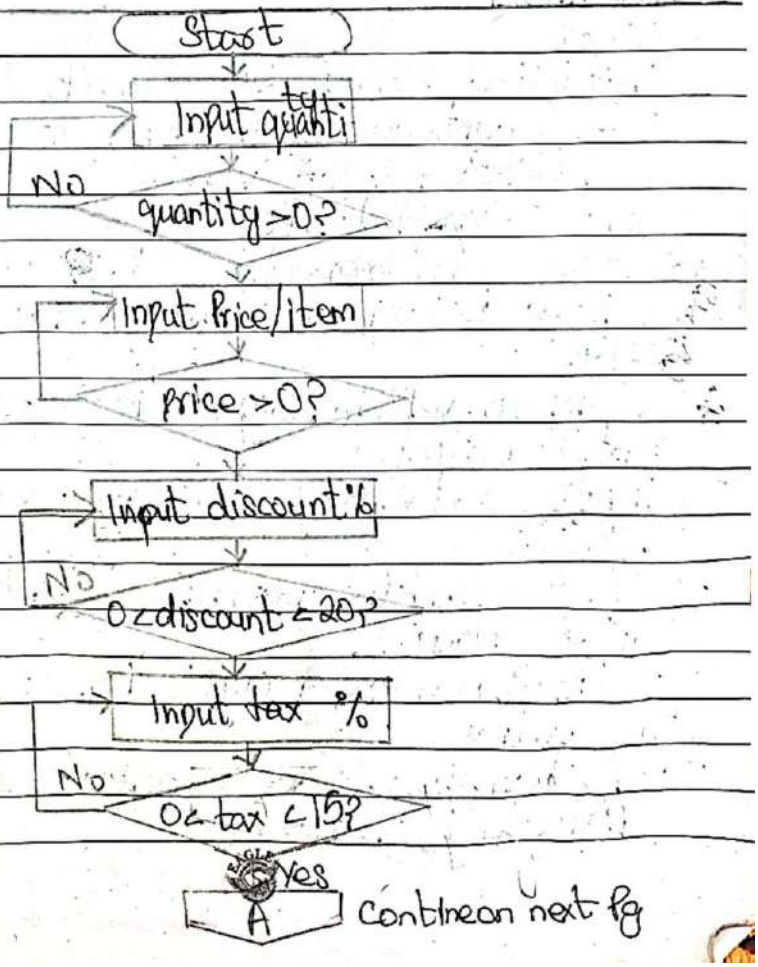
Page No.

PAC Chart

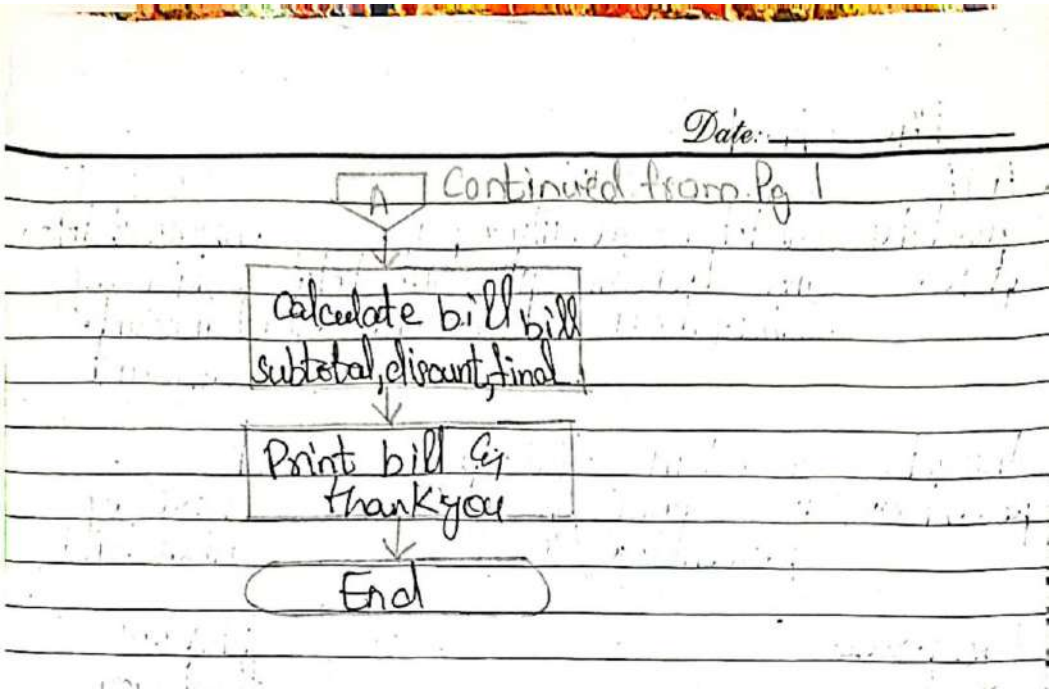
Date: _____

DATA	PROCESSING	OUTPUT
quantity price	sub_total = quantity x price discounted_amount = sub_total - (sub_total x discount_percentage/100)	Display sub_total Display discounted_amount
discount_percentage	final_bill = discounted_amount + (discount_amount x tax_percentage/100)	Display final bill
tax_percentage		Display thank you message;

Flowchart



Date: _____


A
Continued from Pg 1
Calculate bill, bill subtotal, discount, final
Print bill & thankyou
End

Source Code Question 05 Smart Campus Parking and Access Management System

```
#include<stdio.h>

#include<ctype.h>

#include<conio.h>

int main(){

    const int car_space = 1, bike_space = 1, van_space = 2;

    const int max_ZoneA = 20, max_ZoneB = 40, max_ZoneC = 15;

    int exp_vehicles, accepted = 0, rejected = 0, no_of_cars = 0, no_of_vans
= 0, no_of_bikes = 0;

    int total_space = 75;

    int remaining_ZoneA_parking = 20, remaining_ZoneB_parking = 40,
remaining_ZoneC_parking = 15;




printf("+=+=+=
```

```
        goto label1;
    }


    printf("\nIn order to allow you to park your vehicle , kindly answer the
following question....");


    char vehicle_type;
    label2:
    printf("\n\nPlease enter the vehicle type:\tC for CAR\tB for Bike\tV for
VAN :  ");
        vehicle_type = getche();
        vehicle_type = toupper(vehicle_type);


        if(vehicle_type != 'C' && vehicle_type != 'B' && vehicle_type != 'V')
{
            printf("\nInvalid category. Please enter the right vehicle
type[!]\n");
            goto label2;
        }


        char category;
        label3:
        printf("\nEnter the Category:\tF for Faculty\tS for Student\tG for
Guest : ");
        category = getche();
        category = toupper(category);


        if(category != 'F' && category != 'S' && category != 'G') {
            printf("\nInvalid category. Please enter the right vehicle
type[!]\n");
            goto label3;
        }
```

```
        char parking_permit;
        label4:
        printf("\nDo you have valid Permit for Parking (Y/N): ");
        parking_permit = getche();
        parking_permit = toupper(parking_permit);

        if(parking_permit != 'Y' && parking_permit != 'N') {
            printf("\nInvalid Character entered.Please enter one of given
options[!]\n");
            goto label4;
        }

        char emergency;
        label5:
        printf("\nIs your vehicle is an emergency vehicle[!] (Y/N) : \n");
        emergency = getche();
        emergency = toupper(emergency);

        if(emergency != 'Y' && emergency != 'N') {
            printf("\nInvalid Character entered.Please enter one of given
options[!]\n");
            goto label5;
        }

        if(emergency == 'Y') {
            if(category == 'F') {
                if(vehicle_type == 'V') {
                    if(remaining_ZoneA_parking>=van_space) {
                        no_of_vans += 1;
                        accepted += 1;
                        remaining_ZoneA_parking -= van_space;
                        printf("\nZone A assigned to Vehicle due to emergency
irrespective of Parking Permit. Remaining capacity: %d
spaces.\n\n",remaining_ZoneA_parking);
```

```
                    } else {
                        rejected += 1;
                        printf("\nZone A is full.We are really sorry we know
you have emergency vehicle but we can not do anything\n");
                    }
                } else if(vehicle_type == 'C') {
                    if(remaining_ZoneA_parking>=car_space) {
                        no_of_cars += 1;
                        accepted += 1;
                        remaining_ZoneA_parking -= car_space;
                        printf("\nZone A assigned to Vehicle due to emergency
irrespective of Parking Permit. Remaining capacity: %d
spaces.\n\n",remaining_ZoneA_parking);
                    } else {
                        rejected += 1;
                        printf("\nZone A is full.We are really sorry we know
you have emergency vehicle but we can not do anything\n");
                    }
                } else {
                    if(remaining_ZoneA_parking>=bike_space) {
                        no_of_bikes += 1;
                        remaining_ZoneA_parking -= bike_space;
                        accepted += 1;
                        printf("\nZone A assigned to Vehicle due to emergency
irrespective of Parking Permit. Remaining capacity: %d
spaces.\n\n",remaining_ZoneA_parking);
                    } else {
                        rejected += 1;
                        printf("\nZone A is full.We are really sorry we know
you have emergency vehicle but we can not do anything\n");
                    }
                }
            } else if(category == 'S') {
                if(vehicle_type == 'V') {
                    if(remaining_ZoneB_parking>=van_space) {
```

```
                        no_of_vans += 1;

                        remaining_ZoneB_parking -= van_space;

                        accepted += 1;

                        printf("\nZone B assigned to Vehicle due to emergency
irrespective of Parking Permit. Remaining capacity: %d
spaces.\n\n",remaining_ZoneB_parking);

                    } else if(remaining_ZoneC_parking>=van_space) {

                        no_of_vans += 1;

                        remaining_ZoneC_parking -= van_space;

                        accepted += 1;

                        printf("\nZone C assigned to Vehicle due to emergency
irrespective of Parking Permit. Remaining capacity: %d
spaces.\n\n",remaining_ZoneC_parking);

                    } else {

                        rejected += 1;

                        printf("\nZone B and C is full.We are really sorry we
know you have emergency vehicle but we can not do anything\n");

                    }

                } else if(vehicle_type == 'C') {

                    if(remaining_ZoneB_parking>=car_space) {

                        no_of_cars += 1;

                        remaining_ZoneB_parking -= car_space;

                        accepted += 1;

                        printf("\nZone B assigned to Vehicle due to emergency
irrespective of Parking Permit. Remaining capacity: %d
spaces.\n\n",remaining_ZoneB_parking);

                    } else {

                        rejected += 1;

                        printf("\nZone B is full.We are really sorry we know
you have emergency vehicle but we can not do anything\n");

                    }

                } else {

                    if(remaining_ZoneB_parking>=bike_space) {

                        accepted += 1;

                        remaining_ZoneB_parking -= bike_space;
```

```
                        no_of_bikes += 1;

                        printf("\nZone B assigned to Vehicle  due to
emergency irrespective of Parking Permit. Remaining capacity: %d
spaces.\n\n",remaining_ZoneB_parking);

                    } else {

                        rejected += 1;

                        printf("\nZone B is full.We are really sorry we know
you have emergency vehicle but we can not do anything\n");

                    }

                }

            } else {//for guest

                if(vehicle_type == 'V') {

                    if(remaining_ZoneC_parking>=van_space) {

                        accepted += 1;

                        no_of_vans += 1;

                        remaining_ZoneC_parking -= van_space;

                        printf("\nZone C assigned to Vehicle due to emergency
irrespective of Parking Permit. Remaining capacity: %d
spaces.\n\n",remaining_ZoneC_parking);

                    } else {

                        rejected += 1;

                        printf("Zone C is full.We are really sorry we know
you have emergency vehicle but we can not do anything\n");

                    }

                } else if(vehicle_type == 'C') {

                    if(remaining_ZoneC_parking>=car_space) {

                        accepted += 1;

                        no_of_cars += 1;

                        remaining_ZoneC_parking -= car_space;

                        printf("\nZone C assigned to Vehicle due to emergency
irrespective of Parking Permit. Remaining capacity: %d
spaces.\n\n",remaining_ZoneC_parking);

                    } else {

                        rejected += 1;

                        printf("Zone C is full.We are really sorry we know
you have emergency vehicle but we can not do anything\n");
```

```
                    }
                } else {//guest bike
                    if(remaining_ZoneC_parking>=bike_space) {
                        accepted += 1;
                        no_of_bikes += 1;
                        remaining_ZoneC_parking -= bike_space;
                        printf("\nZone C assigned to Vehicle due to emergency
irrespective of Parking Permit. Remaining capacity: %d
spaces.\n\n",remaining_ZoneC_parking);
                    } else {
                        rejected += 1;
                        printf("Zone C is full.We are really sorry we know
you have emergency vehicle but we can not do anything\n");
                    }
                }
            }
        } else {//emergency == 'N'
            if(category == 'F') {
                if(parking_permit == 'Y') {
                    if(vehicle_type == 'V') {
                        if(remaining_ZoneA_parking>=van_space) {
                            accepted += 1;
                            no_of_vans += 1;
                            remaining_ZoneA_parking -= van_space;
                            printf("\nZone A assigned to Vehicle.Have A Good
Day Sir. Remaining capacity: %d spaces.\n\n",remaining_ZoneA_parking);
                        } else {
                            rejected += 1;
                            printf("Zone A is full.We are really sorry\n");
                        }
                    } else if(vehicle_type == 'C') {
                        if(remaining_ZoneA_parking>=car_space) {
                            accepted += 1;
                            no_of_cars += 1;
```

```
                            remaining_ZoneA_parking -= car_space;
                            printf("\nZone A assigned to Vehicle.Have A Good
Day Sir. Remaining capacity: %d spaces.\n\n",remaining_ZoneA_parking);
                        } else {
                            rejected += 1;
                            printf("Zone A is full.We are really sorry\n");
                        }
                    } else {//Faculty Bike
                        if(remaining_ZoneA_parking>=bike_space) {
                            accepted += 1;
                            no_of_bikes += 1;
                            remaining_ZoneA_parking -= bike_space;
                            printf("\nZone A assigned to Vehicle.Have A Good
Day Sir. Remaining capacity: %d spaces.\n\n",remaining_ZoneA_parking);
                        } else {
                            rejected += 1;
                            printf("Zone A is full.We are really sorry\n");
                        }
                    }
                } else {
                    rejected += 1;
                    printf("\nRejected: Faculty member does not have a valid
parking permit.\n");
                }
            } else if(category == 'S') {
                if(parking_permit == 'Y') {
                    if(vehicle_type == 'V') {
                        if(remaining_ZoneB_parking>=van_space) {
                            accepted += 1;
                            no_of_vans += 1;
                            remaining_ZoneB_parking -= van_space;
                            printf("\nZone B assigned to Vehicle .Have A Good
Day Sir. Remaining capacity: %d spaces.\n\n",remaining_ZoneB_parking);
                        } else if (remaining_ZoneC_parking>=van_space) {
```

```
                            accepted += 1;
                            no_of_vans += 1;
                            remaining_ZoneC_parking -= van_space;
                            printf("\nZone C assigned to Vehicle.Have A Good
Day Sir. Remaining capacity: %d spaces.\n\n",remaining_ZoneC_parking);
                        } else {
                            rejected += 1;
                            printf("Zone B and C are full.We are really
sorry\n");
                        }
                    } else if(vehicle_type == 'C') {
                        if(remaining_ZoneB_parking>=car_space) {
                            accepted += 1;
                            no_of_cars += 1;
                            remaining_ZoneB_parking -= car_space;
                            printf("\nZone B assigned to Vehicle.Have A Good
Day Sir. Remaining capacity: %d spaces.\n\n",remaining_ZoneB_parking);
                        } else {
                            rejected += 1;
                            printf("Zone B is full.We are really sorry\n");
                        }
                    } else {//student bikes
                        if(remaining_ZoneB_parking>=bike_space) {
                            accepted += 1;
                            no_of_bikes += 1;
                            remaining_ZoneB_parking -= bike_space;
                            printf("\nZone B assigned to Vehicle .Have A Good
Day Sir. Remaining capacity: %d spaces.\n\n",remaining_ZoneB_parking);
                        } else {
                            rejected += 1;
                            printf("Zone B is full.We are really sorry\n");
                        }
                    }
                } else {
```

```
                    rejected += 1;
                    printf("\nRejected: Student does not have a valid parking
permit.\n");
                }
            } else {//category == 'G'
                if(parking_permit == 'Y') {
                    if(vehicle_type == 'V') {
                        if(remaining_ZoneC_parking>=van_space) {
                            accepted += 1;
                            no_of_vans += 1;
                            remaining_ZoneC_parking -= van_space;
                            printf("\nZone C assigned to Vehicle .Have A Good
Day Sir. Remaining capacity: %d spaces.\n\n",remaining_ZoneC_parking);
                        } else {
                            rejected += 1;
                            printf("Zone C is full.We are really sorry\n");
                        }
                    } else if(vehicle_type == 'C') {
                        if(remaining_ZoneC_parking>=car_space) {
                            accepted += 1;
                            no_of_cars += 1;
                            remaining_ZoneC_parking -= car_space;
                            printf("\nZone C assigned to Vehicle.Have A Good
Day Sir. Remaining capacity: %d spaces.\n\n",remaining_ZoneC_parking);
                        } else {
                            rejected += 1;
                            printf("Zone C is full.We are really sorry\n");
                        }
                    } else {//guest Bike
                        if(remaining_ZoneC_parking>=bike_space) {
                            accepted += 1;
                            no_of_bikes += 1;
                            remaining_ZoneC_parking -= bike_space;
```

```
                            printf("\nZone C assigned to Vehicle .Have A Good
Day Sir. Remaining capacity: %d spaces.\n\n",remaining_ZoneC_parking);
                        } else {
                            rejected += 1;
                            printf("Zone C is full.We are really sorry\n");
                        }
                    }
                } else {
                    rejected += 1;
                    printf("\nRejected: Guest does not have a valid parking
permit.\n");
                }
            }

        }



    int occupied_ZoneA_spaces = max_ZoneA - remaining_ZoneA_parking;
    int occupied_ZoneB_spaces = max_ZoneB - remaining_ZoneB_parking;
    int occupied_ZoneC_spaces = max_ZoneC - remaining_ZoneC_parking;


printf("\n\n+=+=+=
```

```
    printf("Successfully Parked Vehicle Breakdown:\n");

    printf(" - Cars  : %d\n", no_of_cars);

    printf(" - Bikes : %d\n", no_of_bikes);

    printf(" - Vans  : %d\n", no_of_vans);

    printf("-----------------------------------------------------------------
------\n");


    printf("Zone Final Occupancy and Remaining Capacity:\n");

    printf(" - Zone A (Faculty)  : Occupied Spaces = %d | Remaining Capacity
= %d\n", occupied_ZoneA_spaces, remaining_ZoneA_parking);

    printf(" - Zone B (Students) : Occupied Spaces = %d | Remaining Capacity
= %d\n", occupied_ZoneB_spaces, remaining_ZoneB_parking);

    printf(" - Zone C (Visitors) : Occupied Spaces = %d | Remaining Capacity
= %d\n", occupied_ZoneC_spaces, remaining_ZoneC_parking);

    printf("-----------------------------------------------------------------
------\n");


    if (occupied_ZoneA_spaces >= occupied_ZoneB_spaces &&
occupied_ZoneA_spaces >= occupied_ZoneC_spaces) {

        printf("Zone with Highest Occupancy : Zone A (Faculty) with %d
spaces\n", occupied_ZoneA_spaces);

    } else if (occupied_ZoneB_spaces >= occupied_ZoneA_spaces &&
occupied_ZoneB_spaces >= occupied_ZoneC_spaces) {

        printf("Zone with Highest Occupancy : Zone B (Students) with %d
spaces\n", occupied_ZoneB_spaces);

    } else {

        printf("Zone with Highest Occupancy : Zone C (Visitors) with %d
spaces\n", occupied_ZoneC_spaces);

    }


    int total_occupied = occupied_ZoneA_spaces + occupied_ZoneB_spaces +
occupied_ZoneC_spaces;

    if (total_occupied >= 75) {

        printf("Campus Status                 : The entire campus parking
facility is FULL!\n");

    } else {
```

```
        printf("Campus Status                  : The entire campus parking
facility is NOT full.\n");

    }


printf("+=+=+=
```

Q.5. Smart Campus Parking And access Management System.

Algorithm:

1. Start
2. Initialize ZoneA=20, ZoneB=40, ZoneC=15; accepted=rejected=no_of_cars=no_of_bikes=no_of_vans=0.
3. Input exp_vehicles; if <0 or >75, show error and repeat.
4. Set, i=0
5. While i<exp_vehicles, repeat steps 6-14.
6. Input vehicle_type(C/B/V); repeat until valid
7. Input, category (F/S/G); repeat until valid
8. Input parking_permit (Y/N); repeat until valid.
9. Input, emergency (Y/N); repeat until valid
10. Determine space needed: Car=1, Bike=1, Van=2.

Page No.

Date: _____

1. If emergency = Y determine target zone by category alone (ignore permit)
12. Else, if parking_permit ≠ Y reject for "no valid permit"; otherwise, determine target zone by category.
13. Check if the target zone (with zone C fallback for student vans) has enough remaining space.
14. If yes, accept: deduct space, update the vehicle-type counter and accepted, print zone assigned. If no, reject: update rejected, print "zone full".
15. Increment i.
16. Compute occupied spaces per zone (max − remaining).
17. Print the summary report: totals, vehicle-type breakdown, zone occupancy/remaining, highest occupancy zone, campus "full/not-full" status.
18. Stop.

PSEUDOCODE

```
START
  DISPLAY WELCOME MESSAGE
  SET ZoneA = 20, ZoneB = 40, ZoneC = 15
  SET accepted = 0, rejected = 0
  SET cars = 0, bikes = 0, vans = 0.

  REPEAT
      INPUT exp_vehicles.
```

Date: _____

IF exp_vehicles < 0 OR exp_vehicles > 75
THEN DISPLAY "INVALID, TRY AGAIN".

UNTIL exp_vehicles >= 0 AND exp_vehicles
<= 75

Set i = 0
WHILE i < exp_vehicles DO

REPEAT INPUT vehicle_type UNTIL
vehicle type = 'C' OR 'B' OR 'V'

REPEAT INPUT CATEGORY UNTIL CATEGORY
= 'F' OR 'S' OR 'G'

REPEAT INPUT parking permit UNTIL
parking_permit = 'Y' OR 'N'

REPEAT INPUT emergency UNTIL
emergency = 'Y' OR 'N'

IF emergency = 'Y' THEN
IF category = 'F' THEN
IF vehicle_type = 'V' THEN
IF ZoneA >= 2 THEN accept,
vans += 1, ZoneA -= 2.
ELSE reject
ELSE IF vehicle type = 'C' THEN
IF ZoneA >= 1 THEN accept,
cars += 1, ZoneA -= 1

ELSE reject

Date: _____

ELSE IF vehicle type = 'C' THEN
IF Zone A >= 1 THEN accept, cars += 1,
Zone A -= 1
ELSE reject.
ELSE
IF Zone A >= 1 THEN accept,
bikes += 1, Zone A -= 1.
ELSE reject
ENDIF

ELSE IF category = 'S' THEN
IF vehicle_type = 'V' THEN
IF Zone B >= 2 THEN accept,
vans += 1, Zone B -= 2.
ELSE IF Zone C >= 2 THEN,
vans += 1, Zone C -= 2.
ELSE reject.
ELSE IF vehicle_type = 'C' THEN
IF Zone B >= 1 THEN accept,
cars += 1, Zone B -= 1.
ELSE reject.
ELSE
IF Zone B >= 1 THEN accept,
bikes += 1, Zone B -= 1
ELSE reject.
ENDIF

ELSE // Category = 'G'

IF vehicle_type = 'V' THEN
IF Zone C >= 2 THEN accept,
vans += 1, Zone C -= 2.

Date:

```
        ELSE reject.
    ELSE IF vehicle_type = 'C' THEN
        IF Zone C >= 1 THEN accept,
cars += 1, Zone C -= 1
        ELSE reject
    ELSE
        IF Zone C >= 1 THEN, accept,
bikes += 1, Zone C -= 1

        ELSE reject.
    END IF
ENDIF

ELSE // emergency = 'N'
    IF category = 'F' THEN
        IF parking_permit = 'Y' THEN
            IF vehicle_type = 'V' THEN
                IF Zone A >= 2 THEN accept,
Vans += 1, Zone A -= 2
                ELSE reject.
            ELSE IF vehicle_type = 'C' THEN
                IF Zone A >= 1 THEN accept,
cars += 1, Zone A -= 1
                ELSE reject.
            ELSE
                IF Zone A >= 1 THEN accept,
bikes += 1, Zone A -= 1
                ELSE reject
        END IF
    ELSE
        reject "faculty has no valid permit"
    END IF
```

Date: _____

```
ELSE IF category = 'S' THEN
    IF parking_permit = 'Y' THEN
        IF vehicle_type = 'V' THEN
            IF ZoneB >= 2 THEN
accept, vans+=1, ZoneB-=2
            ELSE IF ZoneC >= 2 THEN
accept, vans+=1, ZoneC-=2
            ELSE reject
        ELSE IF vehicle_type = 'C' THEN
            IF ZoneB >= 1 THEN accept,
cars+=1, ZoneB-=1
            ELSE reject
        ELSE
            IF ZoneB >= 1 THEN accept,
bikes+=1, ZoneB-=1
            ELSE reject.
        ENDIF
    ELSE
        reject: "Student has no valid permit"
    ENDIF

ELSE //Category = 'G'
    IF parking_permit = 'Y' THEN
        IF vehicle_type = 'V' THEN
            IF ZoneC >= 2 THEN accept,
vans+=1, ZoneC-=2
            ELSE reject.
        ELSE IF vehicle_type = 'C' THEN
            IF ZoneC >= 1 THEN accept,
cars+=1, ZoneC-=1
            ELSE reject.
        ELSE
```

Date: _____

IF ZoneC >= 1 THEN accept, bikes += 1,
ZoneC -= 1
ELSE reject
ENDIF
ELSE
reject "Guest has no valid permit"
ENDIF
ENDIF
ENDIF

IF Accepted THEN Accepted = Accepted + 1,
DISPLAY Zone assigned message
ELSE, rejected = rejected + 1, DISPLAY
rejection message.
ENDIF
i = i + 1
ENDWHILE

occupiedA = 20 - ZoneA
occupiedB = 20 - ZoneB
occupiedC = 15 - ZoneC

DISPLAY SUMMARY REPORT: totals,
accepted, rejected, cars/bikes/vans
breakdown, occupancy and remaining
capacity per zone, zone with highest
occupancy, campus full/not full status.

END.

IPO Chart

Date: _____

Input	Processing	Module	Output
exp_vehicles	Enter exp_vehicles	1000	
	if exp_vehicles<0 or >75, goto 1000	1100	
	Set i=0	2000	
	if i<exp_vehicles, goto 4000, else goto 9000	3000	
vehicle_type	Enter vehicle_type(C/B/V)	4000	
	if invalid goto 4000	4100	
category	Enter category (F/S/G)	5000	
	if invalid goto 5000	5100	
parking_permit	Enter parking_permit(Y/N)	6000	
	if invalid goto 6000	6100	
emergency	Enter emergency(Y/N)	7000	
	if invalid goto 7000	7100	
	if emergency=Y, zone=Zone for(Category), ignore permit; else if permit=N, goto 8500; else zone=Zonefor (Category)	8000	
	if zone has space, accept, deduct space, update counters, display zone assigned	8100	Zone assigned message
	else reject. display zone full, rejected+=1	8200	"Zone full" message

Page No.

Date: _____

	reject: display no valid permit, rejected += 1	8500	"No permit message"
	i = i+1, goto 3000	8600	
	Compute occupied spaces per zone	9000	
	Display summary report	9100	Summary Report
	end	0000	

Problem Analysis Chart (PAC)

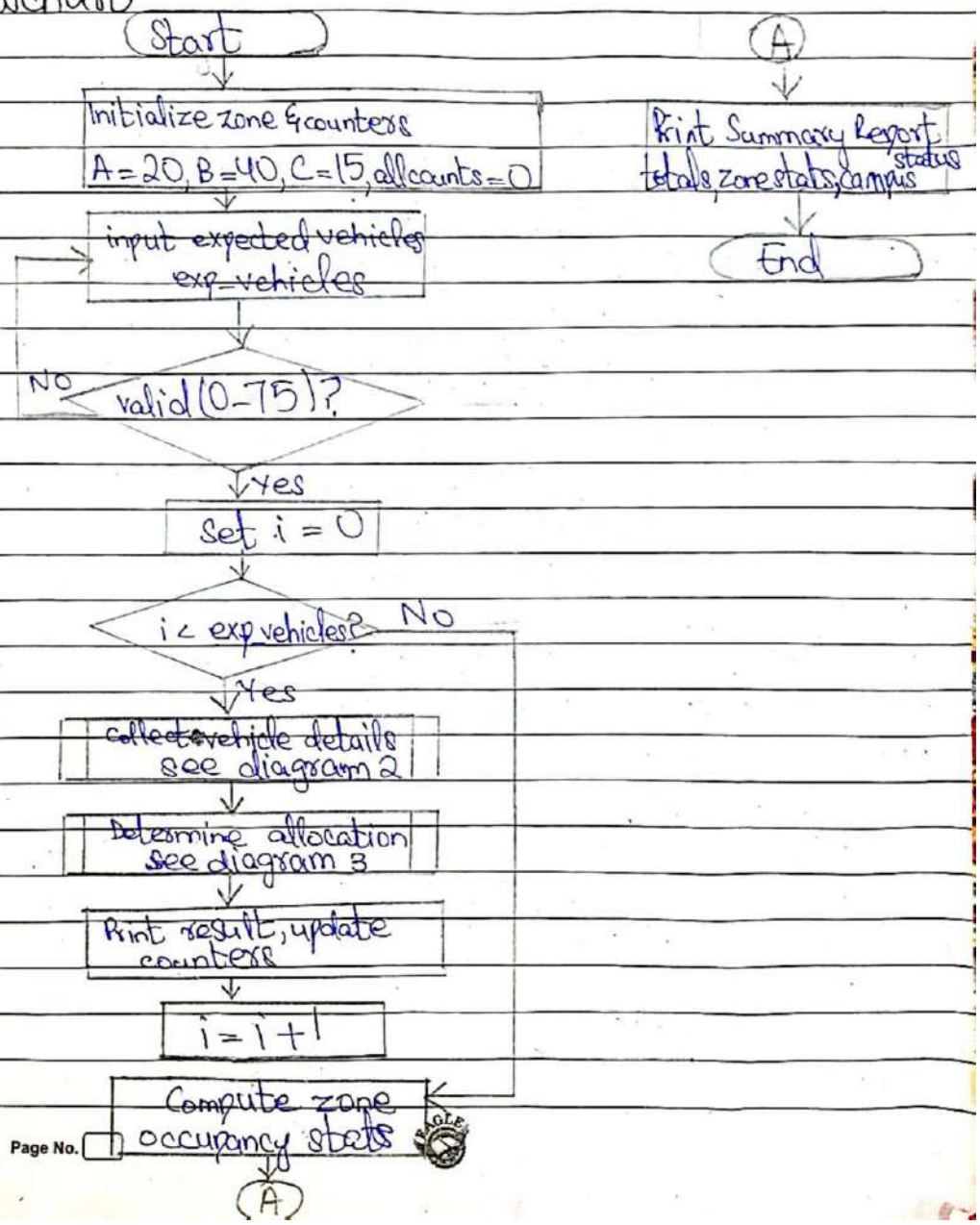
Data	Processing	Output
exp vehicles	space = 2 if vehicle_type = Van, else 1	Display zone assigned or rejection reason; display summary report: totals, breakdown, zone occupancy, highest-occupancy zone, campus status.
vehicle_type	zone = Zone for (category) F → A, S → B (or C for Van overflow), G → C	
category	If emergency = Y, assign zone ignoring permit; else if permit = N, reject; else assign zone	

Page No.

Date:

parking_permit	if remaining_zone >= space, accept and remaining_zone -= space; else reject
emergency	occupied = max_capacity - remaining_capacity (per zone)

Flowchart



Page No.

DIAGRAM 2: Date: _____

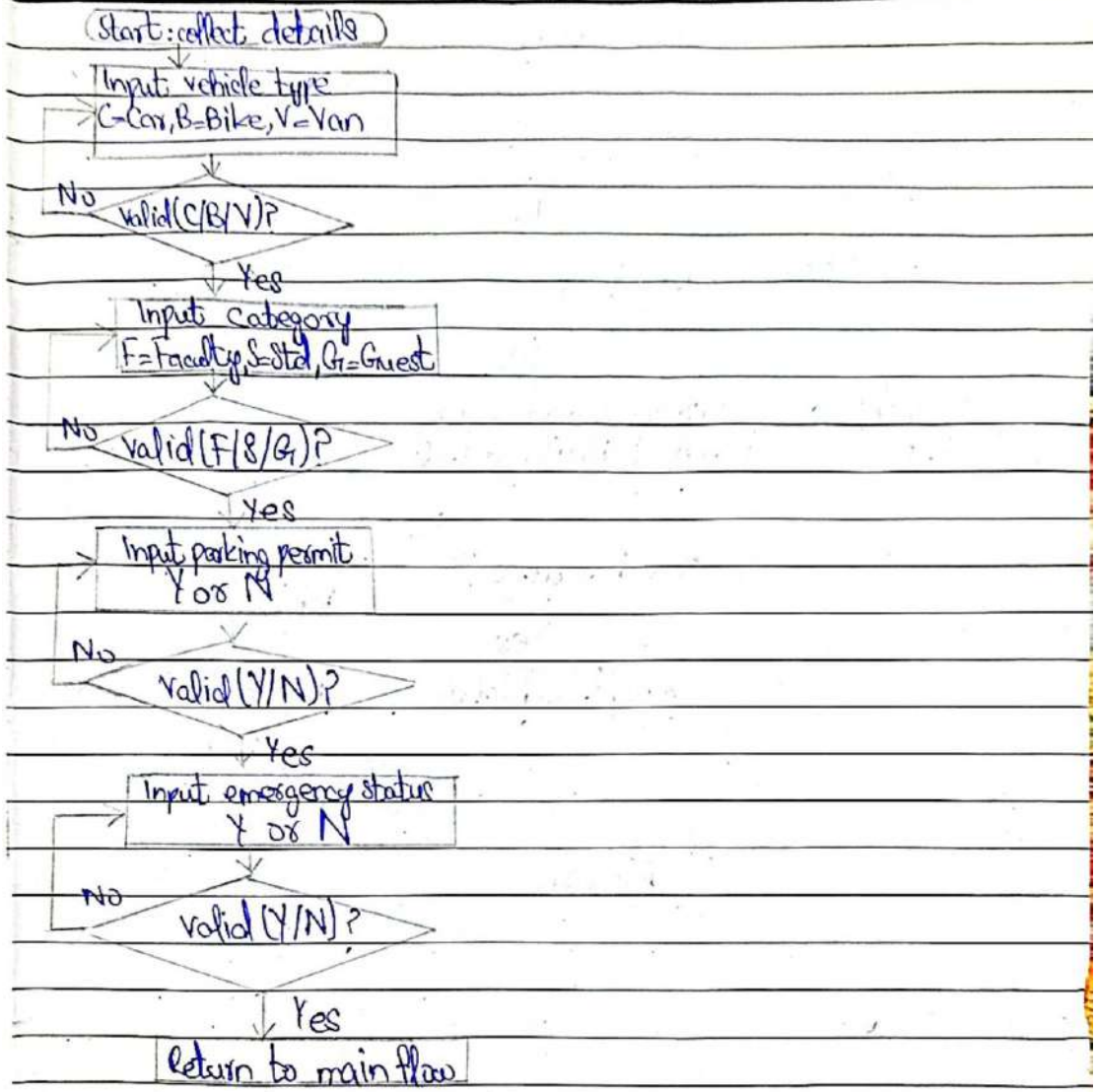
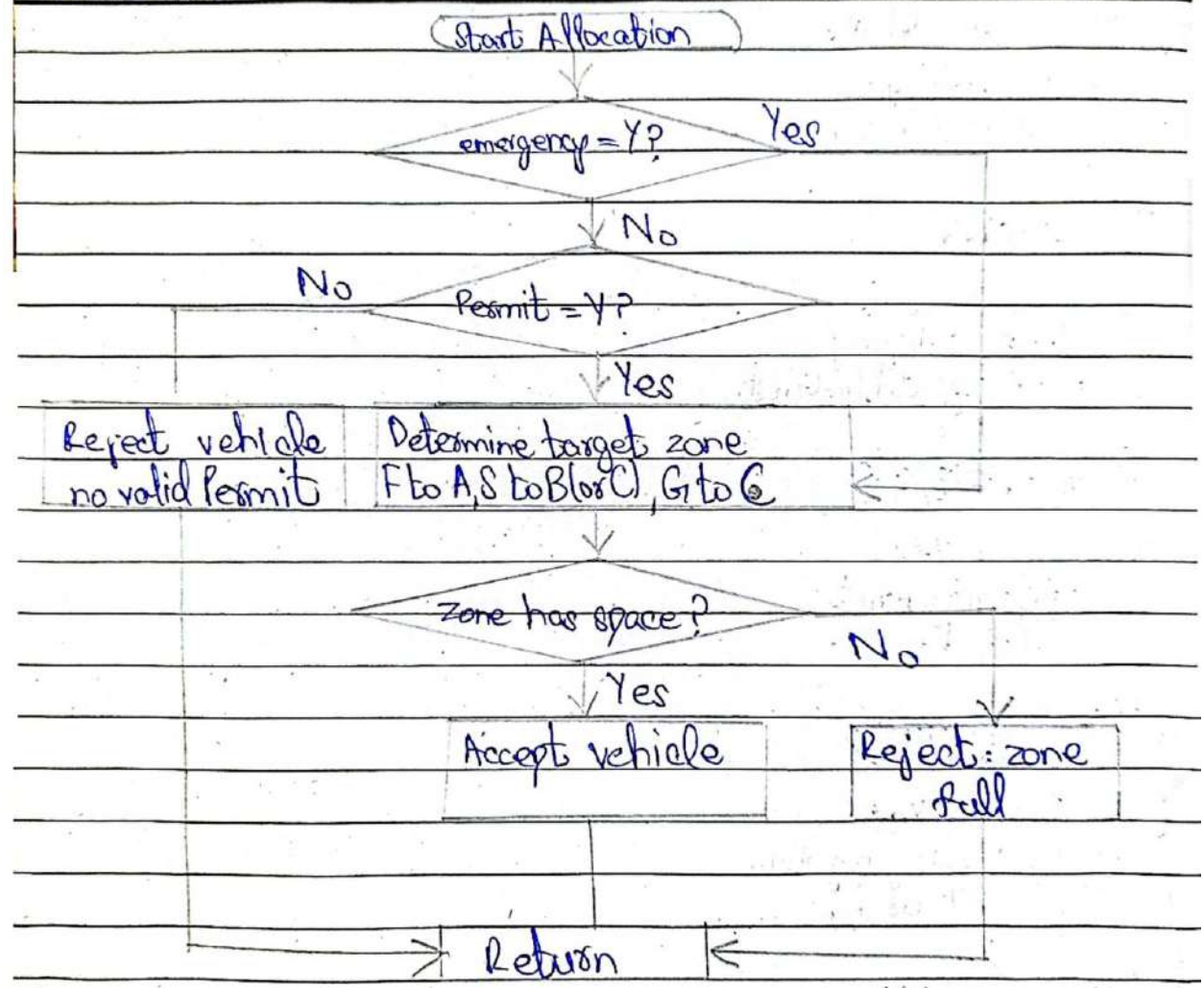


DIAGRAM 3: Date: _____



Source Code Question Question 06 Smart EV Charging and Parking Management System

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
#define PEAK_CHARGING_RATE 50
#define OFF_CHARGING_RATE 35

int main(){
    printf("-------------------------------------------------\n");
    printf("Smart EV Charging and Parking Management System\n");
    printf("-------------------------------------------------\n");

    char vehicle_type;
    label1:
    printf("\nEnter Vehicle Type : E = Electric Vehicle, H = Hybrid Vehicle :
");
    vehicle_type = getche();
    vehicle_type = toupper(vehicle_type);

    if(vehicle_type != 'E' && vehicle_type!= 'H') {
        printf("\nInvalid Input.Enter again[!]\n");
        goto label1;
    }

    int current_battery_percentage;
    label2:
    printf("\nEnter battery Percentage (State of Charge (SOC)) %% : ");
    scanf("%d", &current_battery_percentage);
```

```
    if(current_battery_percentage<=0 || current_battery_percentage>100) {
        printf("\nInvalid Input.Enter again[!]\n");
        goto label2;
    }

    int req_charging_level;

    label3:
    printf("\nEnter Required Charging Level As Percentage %% : ");
    scanf("%d", &req_charging_level);

    if(req_charging_level<=0 || req_charging_level > 100) {
        printf("\nInvalid Input.Enter again[!]\n");
        goto label3;
    }

    float parking_duration;
    label4:
    printf("\nEnter expected parking duration in HOURS (use \"point(.)\" )
for minutes .Ex 23.25 : ");
    scanf("%f", &parking_duration);

    if(parking_duration<0) {
        printf("\nInvalid Input.Enter again[!]\n");
        goto label4;
    }

    float time;

    label5:
    printf("\nEnter Current time in 24-hour format (use \"point(.)\" ) for
minutes .Ex 23.25 : ");
```

```
scanf("%f", &time);

if(time<0 || time>24) {
    printf("\nInvalid Input.Enter again[!]\n");
    goto label5;
}

char parking_membership;
label6:
printf("\nDid you have the parking membership (Y/N) : ");
parking_membership = getche();
parking_membership = toupper(parking_membership);

if(parking_membership != 'Y' && parking_membership != 'N') {
    printf("\nInvalid Input.Enter again[!]\n");
    goto label6;
}

char disabled_person;
label7:
printf("\nDid you have a disabled person priority (Y/N) : ");
disabled_person = getche();
disabled_person = toupper(disabled_person);

if(disabled_person != 'Y' && disabled_person != 'N') {
    printf("\nInvalid Input.Enter again[!]\n");
    goto label7;
}

char charging_station_available;
label8:
printf("\nIs the charging station is currently available (Y/N) ? : ");
```

```
    charging_station_available = getche();
    charging_station_available = toupper(charging_station_available);


    if(charging_station_available != 'Y' && charging_station_available !=
'N') {
        printf("\nInvalid Input.Enter again[!]\n");
        goto label8;
    }


    int emergency_priority = 0,disabled_priority = 0,simple_no_priority =
0,charging_assigned = 1;


    if(charging_station_available == 'N') {
        if(vehicle_type == 'H') {
            charging_assigned = 0;
            printf("\nCharging Unavailable,Parking Only\n");
        } else {
            charging_assigned = 0;
            printf("\nNo chargine slot Avialable\n");
        }
    } else {//charging station available == 'Y'
        if(vehicle_type == 'E') {
            if(req_charging_level<=current_battery_percentage) {
                charging_assigned = 0;
                printf("\nNo Charging Required\n");
            } else {
                if(current_battery_percentage<=15 && req_charging_level>=80)
{
                    emergency_priority = 1;
                printf("\nemergency Charging Priority\n");
                } else if(disabled_person == 'Y' || (parking_membership ==
'Y' && current_battery_percentage<=30)) {
                        disabled_priority = 1;
                        printf("\nPriority Charging\n");
```

```
                } else {
                        simple_no_priority = 1;
                        printf("\nAssign normal charging\n");
                    }
                }
            } else  {//vehicle_type='H'
            if(current_battery_percentage<40) {
                if(req_charging_level<=current_battery_percentage) {
                    charging_assigned = 0;
                    printf("\nNo charging required");
                } else if(current_battery_percentage<=15 &&
req_charging_level>=80) {
                    emergency_priority = 1;
                    printf("\nEmergency Condition Priority");
                } else if(disabled_person == 'Y' || (parking_membership ==
'Y' && current_battery_percentage<=30)) {
                    disabled_priority = 1;
                    printf("\nAssign Priority Charging");
                } else {
                    simple_no_priority = 1;
                    printf("\nNormal Charging");
                }
            } else {
                printf("\nVehicle does not qualify for EV charging.");
            }
        }
    }

    double charging_bill = 0;
    float charging_discount = 0;
    int required_charging;

    if(req_charging_level <= current_battery_percentage) {
```

```
    required_charging = 0;
    } else {
    required_charging = req_charging_level - current_battery_percentage;
    }

    if(charging_assigned != 0) {
        if(time>=17 && time<=22) {//peak
        charging_bill = PEAK_CHARGING_RATE * required_charging;
        if(parking_membership == 'Y' && emergency_priority !=1) {
            charging_discount = charging_bill * 0.10;
            charging_bill = charging_bill - charging_discount;
            printf("\n10% peak membership discount applied[!]");
        }
    } else {//off
        charging_bill = OFF_CHARGING_RATE * required_charging;
        if(parking_membership == 'Y' && emergency_priority != 1) {
            charging_discount = charging_bill*0.20;
            charging_bill = charging_bill - charging_discount;
            printf("\nYou have got 20%% discount because you are a
Member[!]\n");
            }
        }
    }
    float parking_bill = 0;
    float parking_discount = 0;

    if(disabled_person == 'Y') {
        parking_bill = 0;
        parking_discount = 0;
        printf("\nYOU GOT FREE PARKING");
    } else {
        if(parking_duration <= 2) {
```

```
            parking_bill = 200;
        } else if(parking_duration > 2 && parking_duration <= 5) {
            parking_bill = 400;
        } else { // more than 5 hrs
            parking_bill = 700;
        }

        if(parking_membership == 'Y') {
            parking_discount = parking_bill * 0.20; // Fixed multiplier bug
here
            parking_bill = parking_bill - parking_discount;
            printf("\n20% discount applied to parking due to membership[!]");
        }
    }

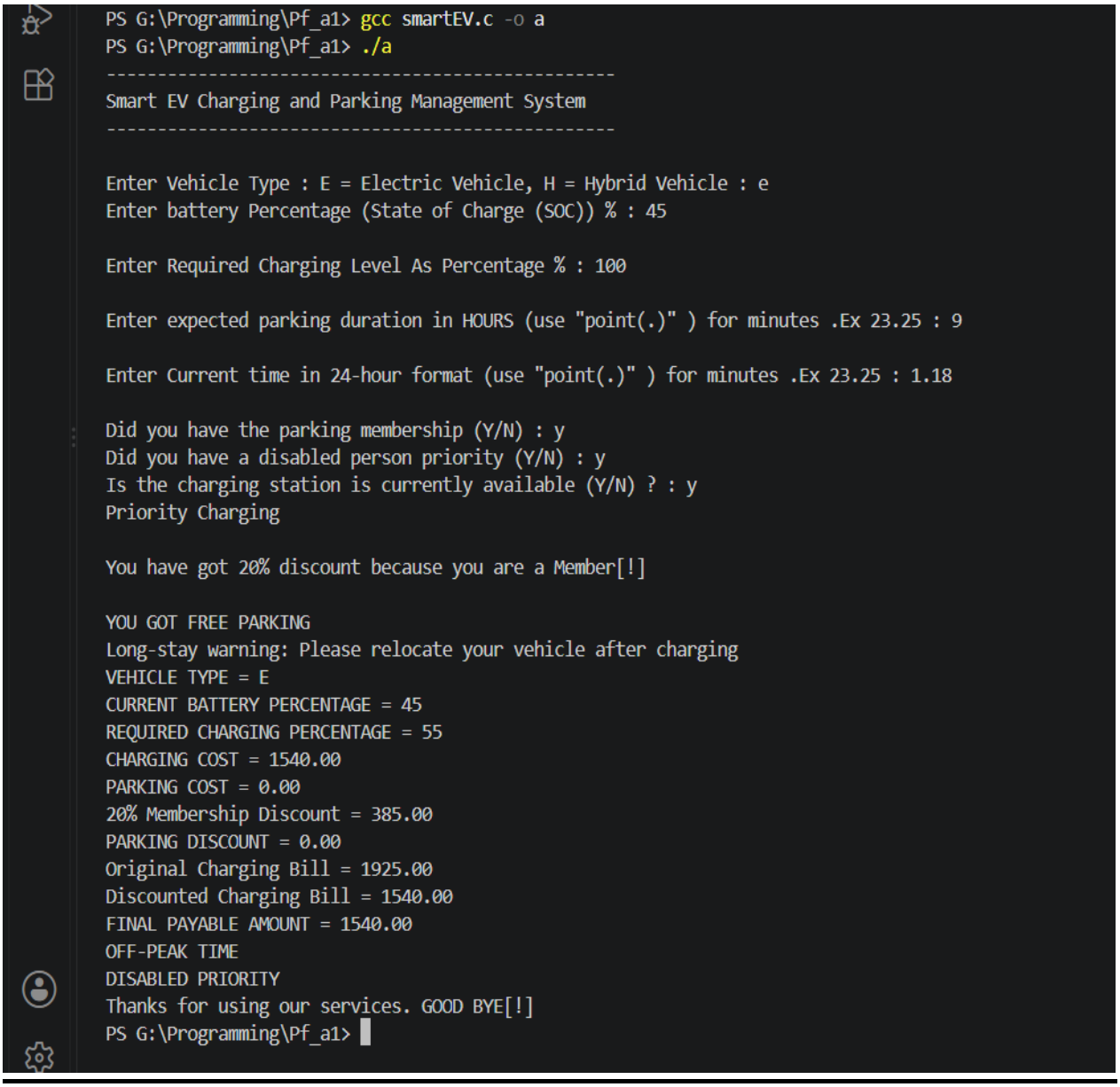
    if(parking_duration > 8) {
        printf("\nLong-stay warning: Please relocate your vehicle after
charging");
    } else {
        printf("\nStandard parking duration");
    }
    double final_amount = parking_bill + charging_bill;
    printf("\nVEHICLE TYPE = %c", vehicle_type);
    printf("\nCURRENT BATTERY PERCENTAGE = %d", current_battery_percentage);
    printf("\nREQUIRED CHARGING PERCENTAGE = %d", required_charging);
    printf("\nCHARGING COST = %.2lf", charging_bill);
    printf("\nPARKING COST = %.2f", parking_bill);
    printf("\n20%% Membership Discount = %.2lf", charging_discount);
    printf("\nPARKING DISCOUNT = %.2f", parking_discount);
    printf("\nOriginal Charging Bill = %.2lf", charging_bill +
charging_discount);
    printf("\nDiscounted Charging Bill = %.2lf", charging_bill);
    printf("\nFINAL PAYABLE AMOUNT = %.2lf", final_amount);
```

```
    if(time>=17 && time<=22) {
        printf("\nPEAK TIME");
    } else {
        printf("\nOFF-PEAK TIME");
    }

    if(emergency_priority == 1) {
        printf("\nEMERGENCY PRIORITY");
    } else if(disabled_priority == 1) {
        printf("\nDISABLED PRIORITY");
    } else if(simple_no_priority == 1) {
        printf("\nSIMPLE PRIORITY(NO PRIORITY)");
    } else {
        printf("\nCHARGING IS NOT EVEN ASSINED / PARKING ONLY");
    }

    printf("\nThanks for using our services. GOOD BYE[!]\n");
    return 0;
}
```

OUTPUT SCREENSHOT:



```
PS G:\Programming\Pf_a1> gcc smartEV.c -o a
PS G:\Programming\Pf_a1> ./a
------------------------------------------------
Smart EV Charging and Parking Management System
------------------------------------------------

Enter Vehicle Type : E = Electric Vehicle, H = Hybrid Vehicle : e
Enter battery Percentage (State of Charge (SOC)) % : 45

Enter Required Charging Level As Percentage % : 100

Enter expected parking duration in HOURS (use "point(.)" ) for minutes .Ex 23.25 : 9

Enter Current time in 24-hour format (use "point(.)" ) for minutes .Ex 23.25 : 1.18

Did you have the parking membership (Y/N) : y
Did you have a disabled person priority (Y/N) : y
Is the charging station is currently available (Y/N) ? : y
Priority Charging

You have got 20% discount because you are a Member[!]

YOU GOT FREE PARKING
Long-stay warning: Please relocate your vehicle after charging
VEHICLE TYPE = E
CURRENT BATTERY PERCENTAGE = 45
REQUIRED CHARGING PERCENTAGE = 55
CHARGING COST = 1540.00
PARKING COST = 0.00
20% Membership Discount = 385.00
PARKING DISCOUNT = 0.00
Original Charging Bill = 1925.00
Discounted Charging Bill = 1540.00
FINAL PAYABLE AMOUNT = 1540.00
OFF-PEAK TIME
DISABLED PRIORITY
Thanks for using our services. GOOD BYE[!]
PS G:\Programming\Pf_a1>
```

(Q6) Smart EV Charging & Parking Management System

Date:

Algorithm:

1. Start
2. Display welcome banner
3. Input vehicle_type (E/H); repeat until valid
4. Input current_battery_percentage; if not in [0,100], repeat
5. Input req_charging_level; if not in [0,100], repeat.
6. Input parking_duration; if <0, repeat.
7. Input time (24-hour); if not in [0,24], repeat.
8. Input parking_membership (Y/N); repeat until valid
9. Input disabled_person (Y/N); repeat until valid.
10. Input charging_station_available (Y/N); repeat until valid.
11. If station is unavailable, set charging_assigned = 0 and display an unavailability message.
12. Else, determine charging eligibility and priority.

- For an EV or for a Hybrid with battery <40%: if req_charging_level ≤ current_battery_percentage, no charging needed. else check emergency (battery ≤ 15 and req ≥ 80), then priority (disabled = Y or member = Y and battery ≤ 30), else normal charging.
- For a Hybrid with battery ≥ 40%, the vehicle does not qualify for charging.

13. Compute required_charging = max(0, req_charging_level − current_battery_percentage)

14. If charging was assigned, compute charging bill using the peak (17:00–22:00) or off-peak rate × required_charging; apply a 10% (peak) or 20% (off-peak) membership discount if the customer is a member and not on emergency priority.

15. If disable_person = Y, parking is free; otherwise compute parking_bill from the duration tiers (≤2h=200, ≤5h=400, >5h=700) and apply a 20% membership discount if applicable.

16. If parking_duration > 8, display a long-stay warning.

17. Compute final_amount = parking_bill + charging_bill

18. Display all bill details: Vehicle type, battery %, required charging %, charging cost, parking cost, discounts, final amount, peak/off-peak label and priority label.

19. Stop.

Date: Module Output

Input	Processing		
vehicle_type	Enter vehicle_type (E/H)	1000	
	If invalid go to 1000	1100	
current_battery_percentage	Enter current_battery_percentage	2000	
	If not in (0,100] goto 2000	2100	
req_charging_level	Enter req_charging_level	3000	
	if not in (0,100], go to 3000	3100	
parking_duration	Enter parking_duration	4000	
	if <0, goto 4000	4100	
time	Enter time (24-hour)	5000	
	if not in [0,24] goto 5000	5100	
parking_membership	Enter parking_membership (Y/N)	6000	
	if invalid, go to 6000	6100	
disabled_person	Enter disabled_person (Y/N)	7000	
	if invalid, go to 7000	7100	
charging_station_available	Enter charging_station_available(Y/N)	8000	
	if invalid, goto 8000	8100	
	If station unavailable, charging_assigned = 0, display message	9000	
	else determine charging eligibility and priority (E/H<40%/H>=40%)	9100	
	required_charging = MAX(0, req_charging_level - current_battery_percentage)	9200	
	if charging_assigned, charging_bill = rate(peak/off_peak) x required_charging; apply membership discount if not emergency	9300	charging cost

Page No.

if disabled_person = Y, parking_bill = 0; else parking_bill by duration tier; apply membership discount	9400	parking cost
If parking_duration > 8 display long-stay warning	9500	
final_amount = parking_bill + charging_bill	9600	
Display full bill details	9700	
end	0000	

Problem Analysis Chart (PAC)

DATA	PROCESSING	OUTPUT
vehicle_type, current_battery_percentage	required_charging = MAX (0, req_charging_level − current_battery_percentage)	Display charging cost & discount
req_charging_level, parking_duration	Charging_bill = (peak ? 50 : 35) × required_charging, minus 10%/20% membership discount if not emergency.	Display parking cost & discount
time, parking membership	parking_bill = 0 if disabled; else 200/400/700 by duration tier, minus 20% membership discount	Display final amount

disabled_person, charging_station_available	final_amount = parking_bill + charging_bill	Display peak/off-peak label, priority label, long-stay warning if applicable.

PSEUDOCODE

```
START
  DISPLAY welcome_banner
  REPEAT INPUT vehicle_type UNTIL vehicle_type = 'E' OR 'H'
  REPEAT INPUT current_battery_percentage UNTIL 0<current_battery_percentage <=100
  REPEAT INPUT parking_duration UNTIL parking_duration >=0
  REPEAT INPUT req_charging_level UNTIL 0 < req_charging_level <=100
  REPEAT INPUT parking_membership UNTIL parking_membership = 'Y' OR 'N'
  REPEAT INPUT disabled_person UNTIL disabled_person = 'Y' OR 'N'
  REPEAT INPUT time UNTIL 0 <= time <= 24
  REPEAT INPUT charging_station_available UNTIL charging_station_availabe = 'Y' OR 'N'

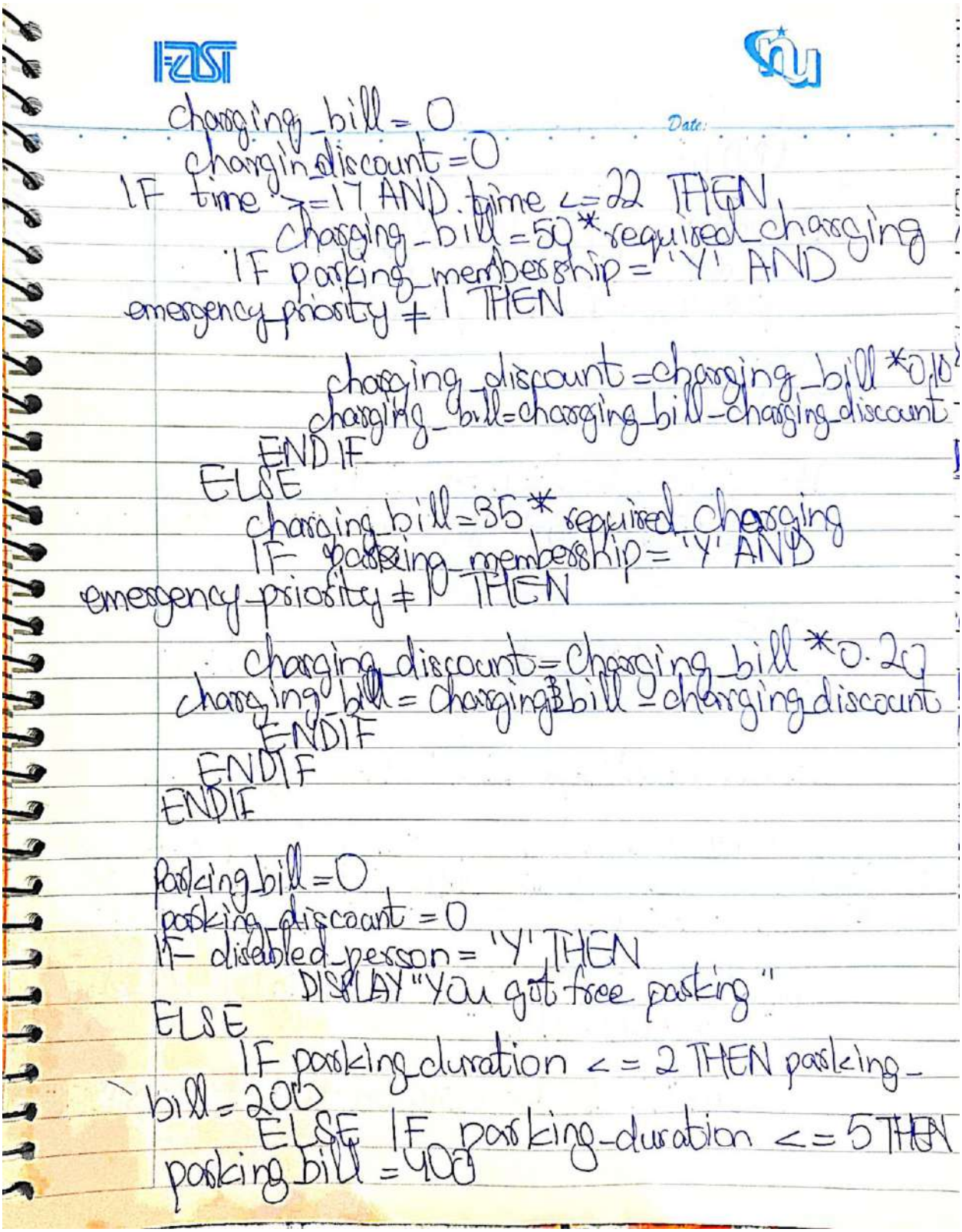
    emergency_priority = 0
    disabled_priority = 0
    simple_no_priority = 0
    charging_assigned = 1
```

IF charging_station_avgilabe = 'N' THEN
charging_assigned = 0
IF vehicle_type = 'H' THEN DISPLAY
"CHARGING UNAVAILABLE, PARKING ONLY"
ELSE DISPLAY "NO Charging slot available"
ENDIF
ELSE
IF vehicle_type = 'E' THEN
IF req_charging_level <= current_battery_
percentage THEN
charging_assigned = 0
DISPLAY "NO Charging Required"
ELSE
IF current_battery_percentage <=15 AND
req_charging_level >=80 THEN

emeggency_priority = 1
DISPLAY "Emeggency Charging Priority".
ELSE IF disabled_person = 'Y' OR
(parking_membership = 'Y' AND current_battery_percentage
<= 30) THEN
disabled_priority = 1
DISPLAY "Priority Charging".
ELSE
Simple_no_priority = 1
DISPLAY "Assign normal charging"
ENDIF
ENDIF
ELSE//Vehicle_type='H'

IF current_battery_percentage <40 THEN
IF req_charging_level <= current_battery_percentage THEN
charging_assigned = 0
DISPLAY "No charging required"
ELSE IF current_battery_percentage <= 15 AND req_charging_level >=80 THEN
emergency_priority=1
DISPLAY "Emergency Condition Priority".
ELSE IF disabled_person = 'Y' OR (parking_membership = 'Y' AND current_battery_percentage <=30) THEN
disabled_priority=1
DISPLAY "Assign Priority Charging".
ELSE
simple_no_priority=1
DISPLAY "Normal Charging".
ENDIF
ELSE
DISPLAY "Vehicle doesn't qualify for EV Charging"
ENDIF
ENDIF
ENDIF

IF req_charging_level <= current_battery_percentage THEN
required_charging = 0
ELSE
required_charging = req_charging_level - current_battery_percentage
ENDIF



```
charging_bill = 0
charging_discount = 0
IF time >= 17 AND time <= 22 THEN
    charging_bill = 50 * required_charging
    IF parking_membership = 'Y' AND
emergency_priority ≠ 1 THEN
        charging_discount = charging_bill * 0.10
        charging_bill = charging_bill - charging_discount
    END IF
ELSE
    charging_bill = 35 * required_charging
    IF parking_membership = 'Y' AND
emergency_priority ≠ 1 THEN
        charging_discount = charging_bill * 0.20
        charging_bill = charging_bill - charging_discount
    ENDIF
  ENDIF
ENDIF

parking_bill = 0
parking_discount = 0
IF disabled_person = 'Y' THEN
        DISPLAY "You got free parking"
ELSE
    IF parking_duration <= 2 THEN parking_
bill = 20
    ELSE IF parking_duration <= 5 THEN
parking_bill = 40
```

ELSE parking_bill = 700
END IF

IF parking_membership = 'Y' THEN
parking_discount = parking_bill * 0.20
parking_bill = parking_bill - parking_discount

END IF
END IF

IF parking_duration > 6 THEN
DISPLAY "Long-stay warning: relocate after charging".

ELSE
DISPLAY "Standard parking duration"
END IF

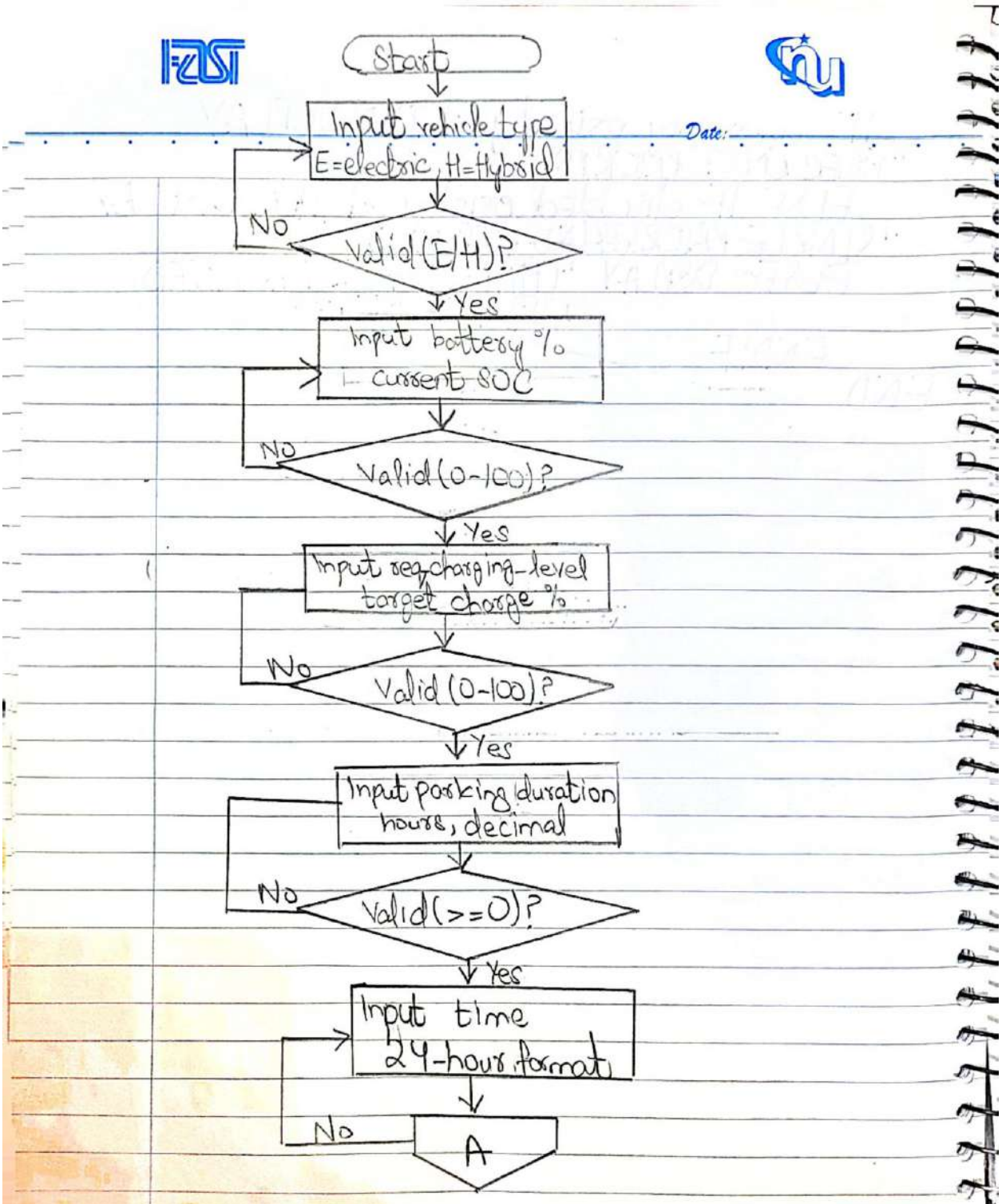
final_amount = parking_bill + charging_bill

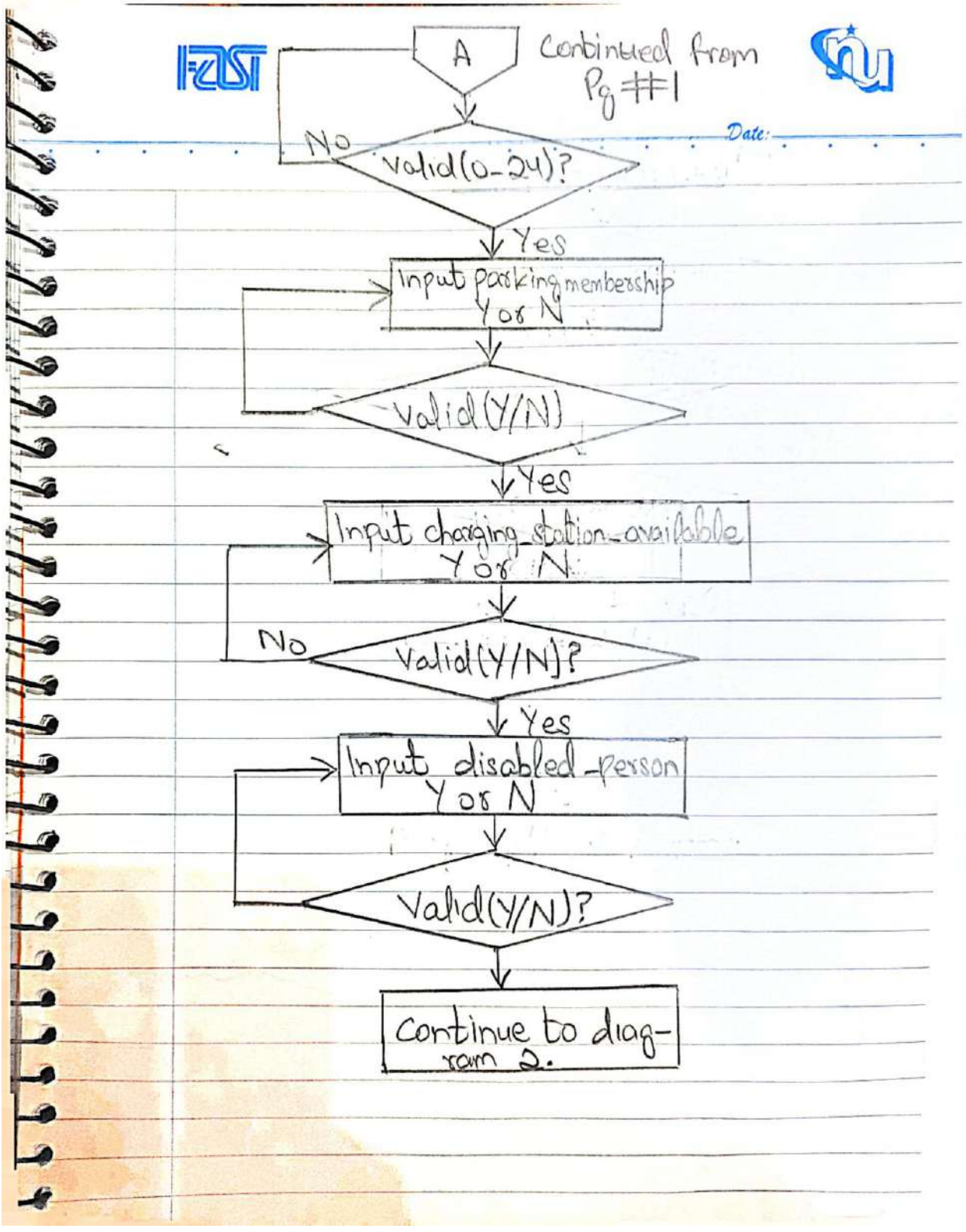
DISPLAY vehicle_type, current_battery_percentage, required_charging, charging_bill, parking_bill, charging_discount, parking_discount, final_amount

DISPLAY "PEAK TIME" if time in [17, 22] else "OFF PEAK TIME"

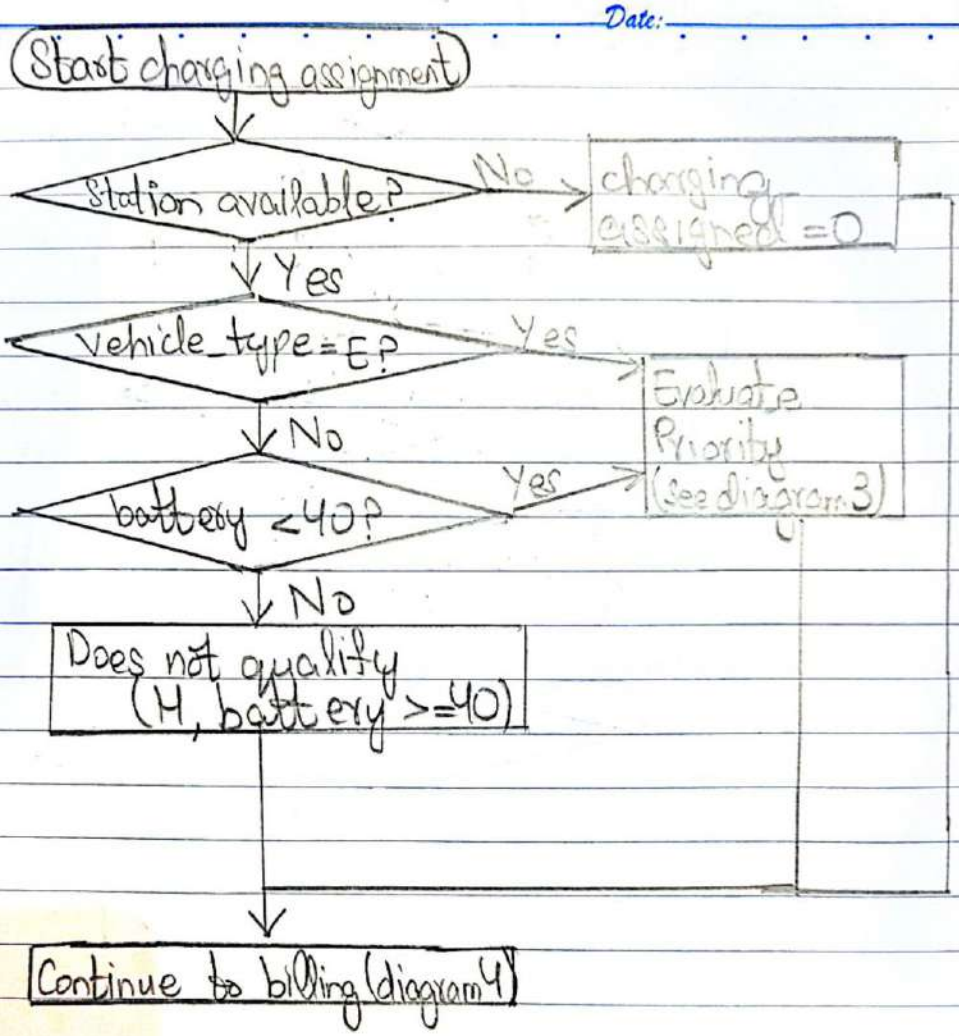
DISPLAY "Thanks for Using Our services. GOOD BYE!"

IF emergency_priority = 1 THEN DISPLAY "EMERGENCY PRIORITY"
ELSE IF disabled_priority = 1 THEN DISPLAY "SIMPLE PRIORITY (NO PRIORITY)"
ELSE DISPLAY "CHARGING NOT ASSIGNED/ PARKING ONLY"
ENDIF
END

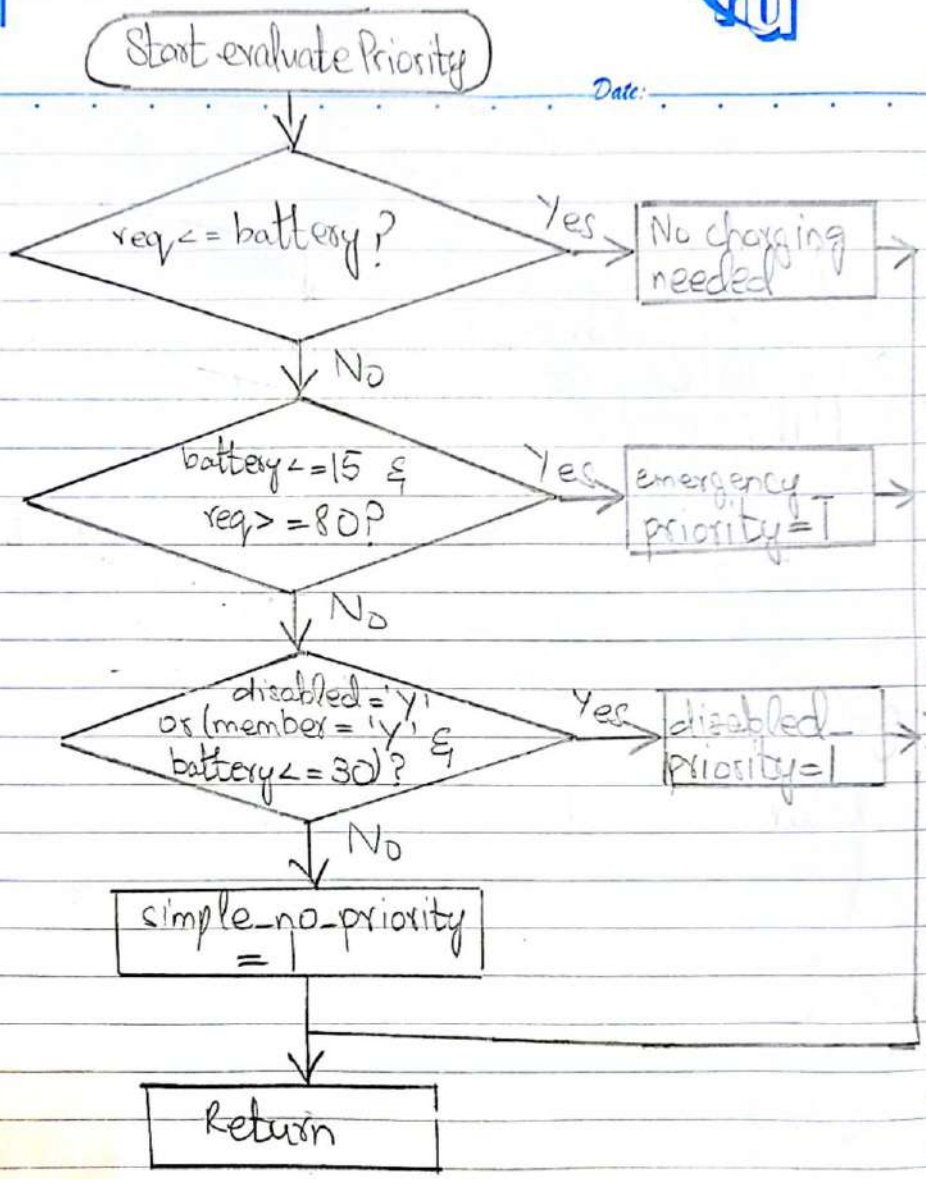

Start
Input vehicle type
E=electric, H=Hybrid
No
Valid (E/H)?
Yes
Input battery %
current SOC
No
Valid (0-100)?
Yes
Input recharging-level
target charge %
No
Valid (0-100)?
Yes
Input parking duration
hours, decimal
No
Valid (>=0)?
Yes
Input time
24-hour format
No
A

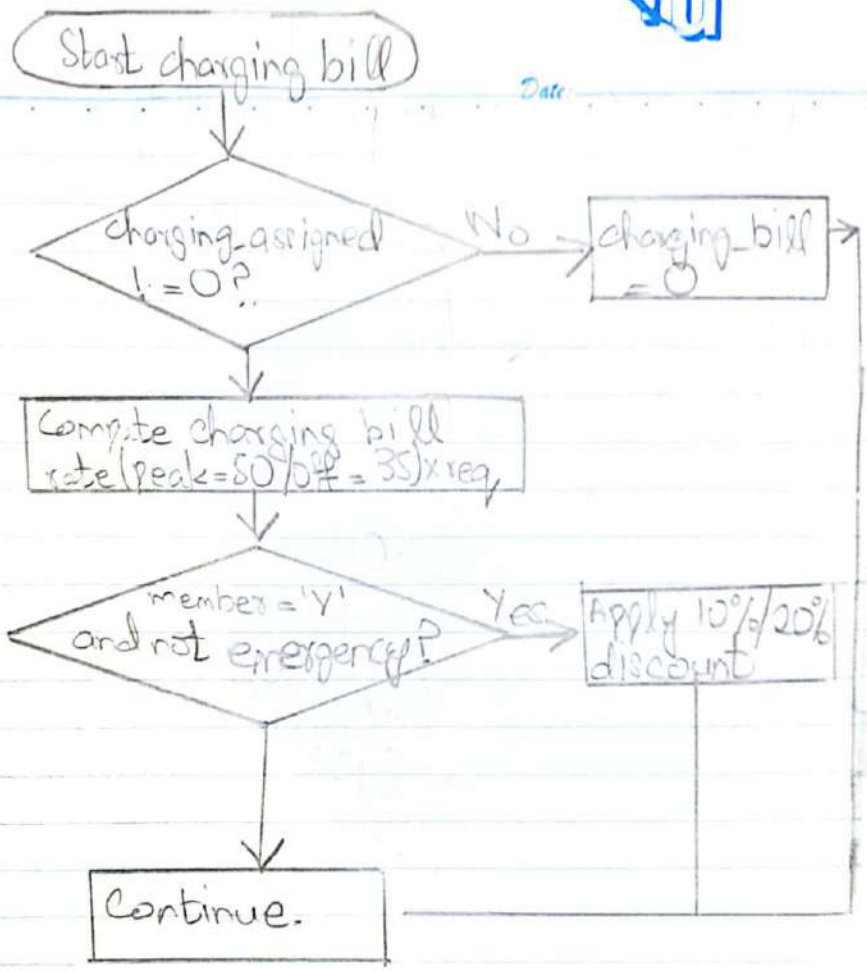

A
Continued from Pg #1
No
valid(0-24)?
Yes
Input parking membership Y or N
Valid(Y/N)
Yes
Input charging_station_available Y or N
No
Valid(Y/N)?
Yes
Input disabled_person Y or N
Valid(Y/N)?
Continue to diagram 2.

Date:

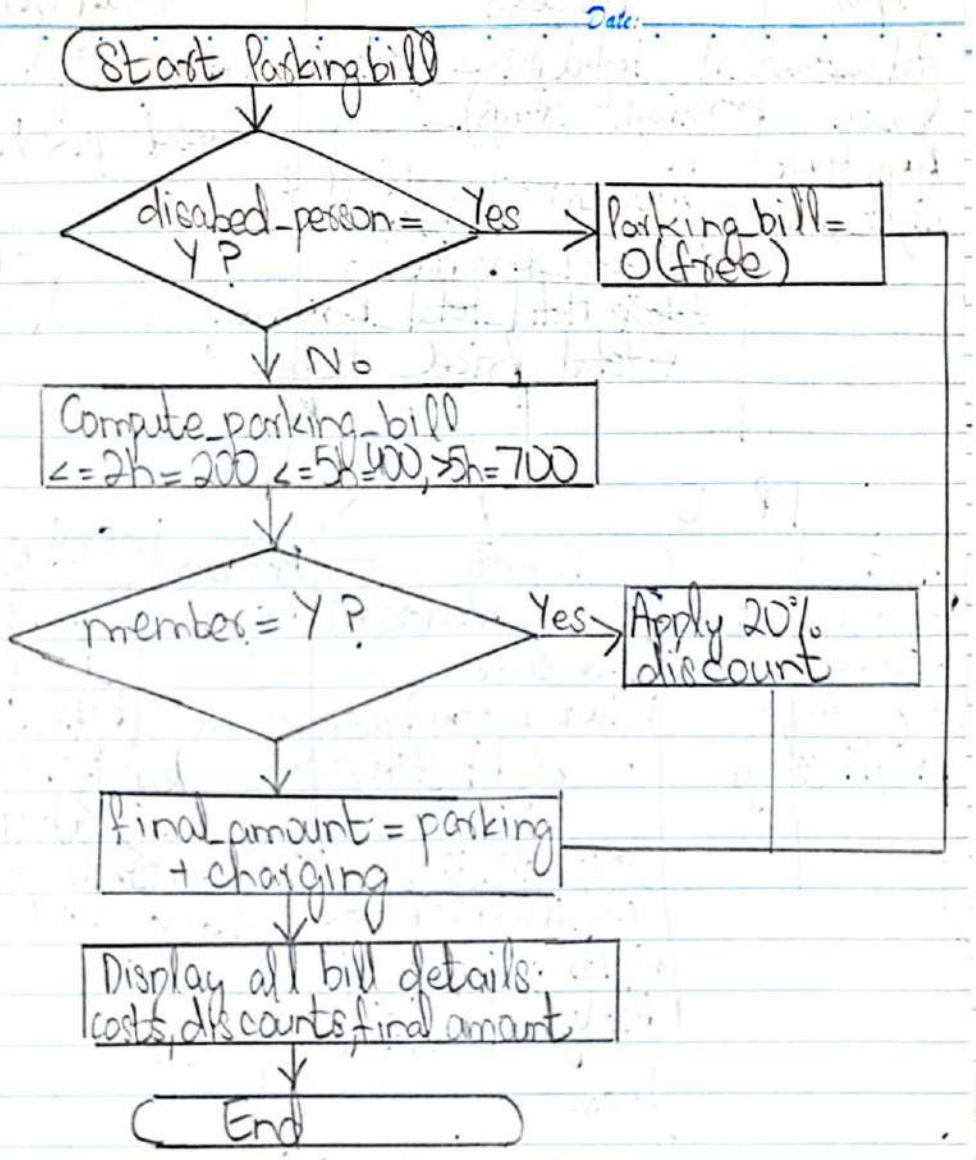

Start charging assignment
Station available?
No
charging assigned = 0
Yes
Vehicle_type = E?
Yes
Evaluate Priority (see diagram 3)
No
battery < 40?
Yes
No
Does not qualify (M, battery >= 40)
Continue to billing (diagram 4)

Date:


Start evaluate Priority
req <= battery ?
Yes
No charging needed
No
battery <= 15 & req >= 80 ?
Yes
emergency priority = 1
No
disabled = 'y' or (member = 'y' & battery <= 30) ?
Yes
disabled_priority = 1
No
simple_no_priority = 1
Return


Start charging bill
charging_assigned != 0 ?
No
charging_bill = 0
Compute charging bill rate (peak = 50/off = 35) x req,
member = 'Y' and not emergency?
Yes
Apply 10%/20% discount
Continue.

Date:


Start Parking bill
disabed_person = Y ?
Yes
Parking_bill = 0(free)
No
Compute_parking_bill
<=2h=200, <=5h=400, >5h=700
member = Y ?
Yes
Apply 20% discount
final_amount = parking + charging
Display all bill details: costs, discounts, final amount
End